

بسم الله الرحمن الرحيم

بسمه تعالی



دانشگاه بناب
گروه علوم کامپیوتر

پایان نامه کارشناسی رشته علوم کامپیوتر

شبکه‌های عصبی آگاه از فیزیک برای حل مسائل
مستقیم و معکوس
معادلات دیفرانسیل جزئی غیرخطی

استاد راهنما: دکتر بابک آذرنوید

پژوهشگر: محمد امیر بابازاد

شماره دانشجویی: ۱۴۰۲۱۳۵۱۰۴

شهریور ۱۴۰۵

کلیه حقوق مادی و معنوی مرتبط با نتایج مطالعات، ابتکارات و نوآوری‌های ناشی از تحقیق موضوع این پایان‌نامه متعلق به دانشگاه بناب است.

تقدیم به

پدر و مادر عزیزم؛
که هرچه دارم از دعای خیر و از خودگذشتگی آنهاست،
و به همه آنان که چراغ دانش را در این سرزمین روشن نگه داشته‌اند.

سپاسگزاری

سپاس بی‌کران خداوند یکتا را که توفیق انجام این پژوهش را به این‌جانب عطا فرمود. بر خود لازم می‌دانم از استاد ارجمند، جناب آقای دکتر بابک آذرنوید که راهنمایی این پایان‌نامه را بر عهده داشتند و در تمام مراحل انجام آن، از انتخاب موضوع تا تدوین نهایی، با حوصله و دقت راهنمایم بودند، صمیمانه قدردانی کنم. بی‌شک بدون رهنمودها و حمایت‌های ایشان، این پژوهش به سرانجام نمی‌رسید. همچنین از همه استادان گروه علوم کامپیوتر دانشگاه بناب که در دوران تحصیل از دانش و تجربه آنان بهره‌مند شدم، تشکر و قدردانی می‌کنم. در پایان، از پدر و مادر مهربانم که همواره مشوق و پشتیبان من در مسیر علم‌آموزی بوده‌اند و از همه دوستان و عزیزانی که به هر شکل مرا در انجام این پژوهش یاری رساندند، سپاسگزارم.

محمد امیر بابازاد

شهریور ۱۴۰۵

چکیده

حل عددی معادلات دیفرانسیل جزئی غیرخطی یکی از مسائل بنیادی در محاسبات علمی و مهندسی به شمار می‌رود. روش‌های عددی کلاسیک با وجود دقت بالا، به تولید شبکه محاسباتی وابسته‌اند و در مواجهه با ابعاد بالا یا هندسه‌های پیچیده با چالش‌های اساسی روبه‌رو می‌شوند. از سوی دیگر، مدل‌های یادگیری عمیق صرفاً داده‌محور نیازمند حجم وسیعی از داده‌های آموزشی بوده و تضمینی برای ارضای قوانین بقای فیزیکی ارائه نمی‌دهند. چارچوب «شبکه‌های عصبی آگاه از فیزیک» (PINN) به‌عنوان رهیافتی نوآورانه برای پیوند دادن معادلات دیفرانسیل حاکم با توانمندی تقریب‌زدن شبکه‌های عصبی عمیق مطرح گردیده است. هدف این پایان‌نامه، مطالعه نظری، پیاده‌سازی محاسباتی و ارزیابی جامع این چارچوب در حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی است.

در این پژوهش، پس از تبیین مبانی نظری معادلات دیفرانسیل، یادگیری عمیق و سازوکار مشتق‌گیری خودکار، فرمول‌بندی مدل‌های زمان‌پیوسته و زمان‌گسسته تشریح شده است. در بخش عملی، مدل زمان‌پیوسته برای معادله غیرخطی برگرز با استفاده از زبان پایتون و کتابخانه جکس پیاده‌سازی گردید. همچنین یک حل‌گر مرجع مستقل مبتنی بر روش طیفی فوریه با گام‌زنی زمانی نمایی مرتبه چهار (ETDRK4) توسعه یافت و صحت عملکرد آن با جواب نیمه‌تحلیلی تبدیل کول-هوفف راستی‌آزمایی شد.

در مسئله مستقیم، میدان جواب تنها با بهره‌گیری از ۲۰۰ نقطه داده اولیه و مرزی با خطای نسبی 2.15×10^{-2} در نرم L_2 نسبت به حل مرجع بازسازی شد؛ تحلیل مکانی نشان داد که بیشینه خطا در لایه باریک جبهه شوک متمرکز است. در مسئله معکوس، ضرایب انتقال و پخش از روی ۵۰۰۰ مشاهده پراکنده در دو سناریوی داده تمیز و آلوده به یک درصد نویز گاوسی شناسایی شدند: در سناریوی بدون نویز ضریب انتقال با خطای نسبی ۸.۸٪ و ضریب پخش با خطای ۳۲.۶٪، و در سناریوی نویزی به‌ترتیب با خطای ۷.۷٪ و ۲۴.۹٪ برآورد گردیدند. یافته‌های این پژوهش مؤید کارایی داده‌ای چشمگیر چارچوب شبکه‌های آگاه از فیزیک و پایداری مناسب آن در برابر اغتشاش داده‌هاست.

کلیدواژه‌ها: شبکه‌های عصبی آگاه از فیزیک، معادلات دیفرانسیل جزئی غیرخطی، مسئله مستقیم، مسئله معکوس، معادله برگرز، مشتق‌گیری خودکار، روش طیفی فوریه.

فهرست اختصارات

در این پایان نامه به منظور سهولت نگارش و اختصار متن، از کوتاه نوشت های استاندارد زیر استفاده شده است:

علامت اختصاری	شرح فارسی
ANN	شبکه عصبی مصنوعی
BC	شرط مرزی
CPU	واحد پردازش مرکزی
GPU	واحد پردازش گرافیکی
IC	شرط اولیه
L-BFGS	روش شبه نیوتنی با حافظه محدود
MLP	پرسپترون چندلایه
MSE	میانگین مربعات خطا
ODE	معادله دیفرانسیل معمولی
PDE	معادله دیفرانسیل جزئی
PINN	شبکه عصبی آگاه از فیزیک
ReLU	واحد خطی یکسوشده
RK	رانگه-کوتا
SSE	مجموع مربعات خطا

فهرست مطالب

پ	سپاسگزاری
ت	چکیده
ث	فهرست اختصارات
ح	فهرست تصاویر
خ	فهرست جداول
۱	۱ مقدمه و کلیات پژوهش
۲	۱-۱ طرح موضوع پژوهش
۳	۲-۱ بیان مسئله
۳	۳-۱ اهداف پژوهش
۴	۴-۱ پرسش‌های پژوهش
۴	۵-۱ روش‌شناسی پژوهش
۵	۶-۱ ساختار پایان‌نامه
۶	۲ مبانی نظری و پیشینه پژوهش
۶	۱-۲ معادلات دیفرانسیل جزئی
۸	۲-۲ روش‌های عددی کلاسیک
۹	۳-۲ شبکه‌های عصبی مصنوعی
۱۱	۴-۲ آموزش شبکه‌های عصبی عمیق
۱۲	۵-۲ مشتق‌گیری خودکار
۱۳	۶-۲ پیشینه پژوهش
۱۴	۷-۲ جمع‌بندی فصل
۱۵	۳ شبکه‌های عصبی آگاه از فیزیک
۱۵	۱-۳ صورت‌بندی کلی مسئله و ایده محوری
۱۷	۲-۳ مدل‌های زمان‌پیوسته برای مسئله مستقیم
۱۸	۳-۳ مدل‌های زمان‌گسسته
۱۸	۴-۳ مسئله معکوس و کشف پارامترهای فیزیکی
۱۹	۵-۳ ملاحظات عملی در پیاده‌سازی و آموزش
۲۰	۶-۳ جمع‌بندی فصل
۲۱	۴ پیاده‌سازی و ارزیابی نتایج
۲۱	۱-۴ بستر محاسباتی و ابزارهای پیاده‌سازی
۲۲	۲-۴ حل‌گر مرجع طیفی و اعتبارسنجی آن

۲۳	حل مستقیم معادله برگرز	۳-۴
۲۵	کشف پارامترهای نامعلوم در مسئله معکوس	۴-۴
۲۶	بحث و تحلیل نتایج	۵-۴
۲۸	جمع‌بندی فصل	۶-۴
۲۹	نتیجه‌گیری و پیشنهادها	۵
۲۹	۱-۵ خلاصه پژوهش	
۳۰	۲-۵ یافته‌ها و دستاوردهای کلیدی	
۳۱	۳-۵ پیشنهادها برای پژوهش‌های آینده	
۳۲	فهرست منابع	
۳۴	واژه‌نامه	
۳۶	پیوست الف: کد پیاده‌سازی محاسباتی	
۴۳	فرم ارزیابی پایان‌نامه	
۴۴	Abstract	

فهرست تصاویر

۱-۲	ساختار شماتیک یک شبکه عصبی پرسپترون چندلایه با دو متغیر ورودی، دو لایه پنهان و یک خروجی اسکالر	۱۰
۱-۳	نمودار بلوکی چارچوب شبکه عصبی آگاه از فیزیک: تعامل شبکه پاسخ، مشتق‌گیری خودکار، شبکه باقیمانده و حلقه بهینه‌سازی	۱۶
۱-۴	توزیع نقاط آموزشی در مسئله مستقیم معادله برگرز: نقاط داده اولیه و مرزی ($N_u = 200$) و نقاط هم‌مکانی باقیمانده ($N_f = 10000$)	۲۳
۲-۴	تاریخچه کاهش تابع هزینه در مسئله مستقیم: همگرایی مرحله اول با الگوریتم آدام و اصلاح نهایی با روش شبه‌نیوتنی L-BFGS	۲۴
۳-۴	مقایسه پروفیل‌های زمانی حل دقیق مرجع و پیش‌بینی شبکه آگاه از فیزیک در سه لحظه $t = 0.25, 0.5, 0.75$	۲۵
۴-۴	نقشه توزیع خطای قدرمطلق $ u_{\text{ref}} - u_{\text{PINN}} $ در دامنه زمانی-مکانی در مسئله مستقیم	۲۵
۵-۴	توزیع مکانی-زمانی ۵۰۰۰ نقطه مشاهده پراکنده در مسئله معکوس؛ طیف رنگی نمایانگر مقدار متغیر حالت $u(t, x)$ است	۲۶

فهرست جداول

۱-۲	مقایسه ویژگی‌های روش‌های عددی کلاسیک در حل معادلات
۹	دیفرانسیل جزئی
۲-۲	توابع فعال‌ساز متداول در شبکه‌های عصبی عمیق
۱۰	
۱-۳	مقایسه تطبیقی مدل‌های زمان‌پیوسته و زمان‌گسسته در چارچوب
۲۰	شبکه‌های آگاه از فیزیک
۱-۴	تنظیمات ساختاری و پارامترهای آموزش مدل زمان‌پیوسته در مسئله
۲۴	مستقیم
۲-۴	مقایسه تطبیقی نتایج حل مستقیم با مقاله مرجع رئیسی و همکاران
۲۶	(۲۰۱۹)
۳-۴	نتایج تخمین پارامترهای مجهول معادله برگرز در مسئله معکوس . .
۲۷	

فصل ۱

مقدمه و کلیات پژوهش

بسیاری از پدیده‌های بنیادی در علوم پایه و مهندسی، از جمله دینامیک سیالات، انتقال حرارت، مکانیک کوانتومی، انتشار امواج و فرایندهای بیوفیزیکی، در قالب معادلات دیفرانسیل جزئی^۱ مدل‌سازی می‌شوند. از این رو، ارائه روش‌های دقیق، کارآمد و محاسبات‌پذیر برای تحلیل و حل این معادلات همواره یکی از ارکان اصلی در حوزه محاسبات علمی بوده است. در دهه‌های اخیر، پیشرفت‌های شگرف در حوزه یادگیری عمیق^۲ تحولات بنیادینی را در پردازش تصویر، پردازش زبان طبیعی و علوم داده رقم زده است. با این وجود، به‌کارگیری مستقیم مدل‌های یادگیری عمیق صرفاً داده‌محور در مسائل مهندسی و فیزیکی با چالش‌های ساختاری مهمی روبه‌روست؛ از جمله نیاز به حجم عظیمی از داده‌های اندازه‌گیری‌شده و برچسب‌دار که در اغلب سامانه‌های مهندسی دستیابی به آن‌ها هزینه‌بر یا ناممکن است. افزون بر این، خروجی این مدل‌ها فاقد تضمین برای سازگاری با اصول و قوانین بنیادین فیزیک (نظیر بقای جرم، تکانه و انرژی) است و ممکن است در دامنه‌های برون‌یابی‌شده به رفتارهای غیرفیزیکی منجر گردد.

به‌منظور غلبه بر این محدودیت‌ها، ادغام دانش فیزیکی حاکم بر سیستم با ساختارهای یادگیری ماشین، رویکرد نوینی را شکل داده است. حاصل این پیوند، خانواده‌ای از مدل‌ها تحت عنوان «شبکه‌های عصبی آگاه از فیزیک»^۳ است که توسط رئیسی و همکارانش در سال ۲۰۱۹ به‌صورت یکپارچه معرفی شد [۱] و با استقبال گسترده جامعه علمی در مرز مشترک هوش مصنوعی و محاسبات علمی روبه‌رو گردید. در این رهیافت، معادله دیفرانسیل حاکم به‌همراه شرایط مرزی و اولیه مستقیماً درون تابع هزینه مدل لحاظ می‌شود.

^۱Partial Differential Equation (PDE)

^۲Deep Learning

^۳Physics-Informed Neural Network (PINN)

در این پایان‌نامه، چارچوب شبکه‌های عصبی آگاه از فیزیک برای حل هر دو دسته مسائل مستقیم (محاسبه میدان جواب) و مسائل معکوس (کشف پارامترهای نامعلوم سیستم) در معادلات دیفرانسیل جزئی غیرخطی مورد ارزیابی قرار می‌گیرد. مسئله محوری انتخابی برای پیاده‌سازی و سنجش در این پژوهش، معادله غیرخطی برگرز^۴ است؛ معادله‌ای کلاسیک که به دلیل ایجاد جبهه‌های شیب‌دار تند و رفتار موج شوک، معیاری استاندارد و چالش‌برانگیز برای سنجش کارایی روش‌های محاسباتی به شمار می‌آید. علاوه بر تحلیل نظری جامع، پیاده‌سازی محاسباتی مدل زمان‌پیوسته با زبان برنامه‌نویسی پایتون و کتابخانه جکس^۵ صورت گرفته و نتایج حاصل با یک حل‌گر طیفی مستقل بر پایه روش فوریه مقایسه و اعتبارسنجی شده است.

۱-۱ طرح موضوع پژوهش

موضوع محوری این پایان‌نامه «شبکه‌های عصبی آگاه از فیزیک برای حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی» است. در مسائل مستقیم^۶، ساختار و پارامترهای معادله دیفرانسیل به‌طور کامل مشخص بوده و هدف اصلی، تعیین توزیع مکانی-زمانی متغیر مجهول با حداقل داده‌های اولیه و مرزی است. در مقابل، در مسائل معکوس^۷، با در دست داشتن تعدادی اندازه‌گیری یا مشاهده پراکنده (که می‌تواند با خطای اندازه‌گیری و نویز همراه باشد)، هدف استخراج و تخمین پارامترهای مجهول حاکم بر فیزیک مسئله است. چارچوب شبکه‌های آگاه از فیزیک هر دو دسته مسئله را در قالبی یکپارچه فرمول‌بندی می‌نماید [۱].

رکن اساسی در این چارچوب، بهره‌گیری از سازوکار مشتق‌گیری خودکار^۸ است. در این شیوه، مشتقات جزئی متغیر وابسته نسبت به متغیرهای مستقل فضا و زمان، بدون نیاز به گسسته‌سازی مکانی و تفاضل محدود، مستقیماً از روی گراف محاسباتی شبکه عصبی و با دقت ماشین محاسبه می‌شوند. این ویژگی امکان ارزیابی پیوسته باقیمانده معادله دیفرانسیل را در قالب یک شبکه باقیمانده فراهم ساخته و شبکه را به سمت یادگیری تابعی سوق می‌دهد که هم‌زمان به داده‌های تجربی نزدیک بوده و ساختار معادله حاکم را نیز برآورده سازد.

^۴Burgers Equation

^۵JAX

^۶Forward Problem

^۷Inverse Problem

^۸Automatic Differentiation

۲-۱ بیان مسئله

روش‌های عددی کلاسیک نظیر روش تفاضل‌های محدود^۹، روش اجزای محدود^{۱۰} و روش‌های طیفی^{۱۱} طی چندین دهه ابزارهای استوار و تثبیت‌شده در حل معادلات دیفرانسیل جزئی بوده‌اند [۱۶]. با این حال، این روش‌ها به تولید شبکه محاسباتی وابستگی شدیدی دارند و در مسائل با ابعاد بالا، با پدیده نفرین ابعاد^{۱۲} و رشد نمایی هزینه محاسباتی روبه‌رو می‌شوند. علاوه بر این، در مسائلی که شرایط هندسی نامنظم یا داده‌های پراکنده و ناکافی وجود دارد، اعمال روش‌های کلاسیک با دشواری‌های تکنیکی همراه است.

در نقطه مقابل، شبکه‌های عصبی استاندارد اگرچه از قابلیت بالایی در برازش توابع غیرخطی برخوردارند، اما در صورت عدم برخورداری از قیدهای ساختاری، به حجم بسیار زیادی از داده‌های آموزشی نیاز داشته و خروجی آن‌ها در مناطقی که فاقد داده آموزشی است، ممکن است دچار خطای برون‌یابی شدید و نقض آشکار قوانین فیزیکی گردد. مسئله اصلی این پژوهش را می‌توان در این پرسش خلاصه کرد: چگونه می‌توان با تلفیق ساختار ریاضی معادلات دیفرانسیل جزئی در فرایند آموزش شبکه‌های عصبی عمیق، سیستمی محاسباتی ایجاد کرد که اولاً بدون نیاز به شبکه‌بندی محاسباتی و با داده‌های اولیه و مرزی محدود بتواند پاسخ دقیق معادلات غیرخطی را بازسازی کند، و ثانیاً توانایی تخمین پارامترهای نامعلوم فیزیکی را از روی داده‌های تجربی پراکنده و نویزی داشته باشد؟ مدل‌های زمان‌پیوسته و زمان‌گسسته شبکه‌های عصبی آگاه از فیزیک [۱] پاسخی ساختاریافته به این نیاز محاسباتی ارائه می‌دهند.

۳-۱ اهداف پژوهش

هدف کلی این پایان‌نامه، تحلیل مبانی نظری، پیاده‌سازی الگوریتمی و ارزیابی تجربی چارچوب شبکه‌های عصبی آگاه از فیزیک در حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی است. اهداف تفصیلی پژوهش عبارت‌اند از:

۱. بررسی و تدوین مبانی نظری معادلات دیفرانسیل جزئی، روش‌های عددی کلاسیک، شبکه‌های عصبی پیش‌خور، روش‌های بهینه‌سازی و تکنیک‌های مشتق‌گیری خودکار؛

⁹Finite Difference Method

¹⁰Finite Element Method

¹¹Spectral Methods

¹²Curse of Dimensionality

۲. تحلیل ساختار مدل‌های زمان‌پیوسته و زمان‌گسسته شبکه‌های آگاه از فیزیک در مسائل مستقیم و معکوس؛
۳. طراحی و توسعه یک حل‌گر مرجع طیفی مستقل بر پایه تبدیل فوریه برای معادله غیرخطی برگرز به‌منظور ایجاد معیار ارزیابی دقیق؛
۴. پیاده‌سازی محاسباتی مدل زمان‌پیوسته برای حل مستقیم معادله برگرز و تحلیل خطای تقریب در مقایسه با حل‌گر مرجع؛
۵. پیاده‌سازی مسئله معکوس برای شناسایی ضرایب نامعلوم معادله برگرز از روی مشاهدات پراکنده و ارزیابی حساسیت الگوریتم در حضور نویز داده‌ها.

۴-۱ پرسش‌های پژوهش

این پژوهش در صدد پاسخ‌گویی به پرسش‌های علمی زیر است:

۱. آیا یک شبکه عصبی با ساختار کم‌عمق تا متوسط قادر است تنها با اتکا بر داده‌های مرزی-اولیه محدود و باقیمانده معادله، پاسخ یک معادله دیفرانسیل جزئی غیرخطی نظیر معادله برگرز را در کل دامنه زمانی-مکانی تقریب بزند؟
۲. فرایند مشتق‌گیری خودکار چگونه باقیمانده فیزیکی را در ساختار تابع هزینه مدل تلفیق می‌نماید؟
۳. خطای عددی حاصل از شبکه عصبی آگاه از فیزیک در مقایسه با روش‌های مرجع طیفی چگونه توزیع می‌شود و چه نواحی از دامنه دارای بیشترین خطا هستند؟
۴. دقت و پایداری تخمین پارامترهای مجهول فیزیکی در مسئله معکوس در شرایط وجود داده‌های پراکنده و آلوده به نویز چگونه رفتار می‌کند؟
۵. مزایا و محدودیت‌های ذاتی مدل‌های زمان‌پیوسته در مقایسه با روش‌های کلاسیک و مدل‌های زمان‌گسسته چیست؟

۵-۱ روش‌شناسی پژوهش

روش‌شناسی به کار رفته در این پایان‌نامه مبتنی بر تلفیق مطالعات تحلیلی-نظری و پیاده‌سازی‌های محاسباتی-تجربی است:

در گام نخست، ادبیات پژوهش و مراجع معتبر در حوزه‌های معادلات دیفرانسیل جزئی [۱۷]، روش‌های عددی [۱۶]، یادگیری عمیق [۱۱] و مشتق‌گیری خودکار [۱۵] مورد مطالعه جامع قرار گرفت و مبانی نظری شبکه باقیمانده و توابع هزینه تلفیقی استخراج

شد.

در گام دوم، معادله برگرز یک‌بعدی با شرایط اولیه سینوسی و شرایط مرزی همگن به دلیل رفتار موج شوک غیرخطی به عنوان مسئله آزمون انتخاب شد. برای ایجاد حل دقیق، یک حل گر مرجع طیفی فوریه با روش گام زنی زمانی مرتبه چهار نمایی (ETDRK4) توسعه یافت و صحت آن با جواب نیمه تحلیلی کول-هوپف [۱۸] راستی آزمایی گردید.

در گام سوم، مدل زمان پیوسته شبکه عصبی آگاه از فیزیک با استفاده از معماری پرسپترون چندلایه و بهینه سازی ترکیبی دومرحله ای (آدام و سپس L-BFGS) در زبان پایتون با کتابخانه محاسباتی جکس پیاده سازی شد.

در گام چهارم، آزمایش های عددی شامل حل مسئله مستقیم، کشف ضرایب در مسئله معکوس در سناریوهای داده تمیز و نویزی اجرا شده و معیارهای کمی نظیر خطای نسبی در نرم L_2 و تاریخچه همگرایی توابع هزینه مورد تحلیل قرار گرفت.

۶-۱ ساختار پایان نامه

فصول این پایان نامه به صورت زیر تدوین شده است:

- **فصل دوم (مبانی نظری و پیشینه پژوهش):** مروری بر مفاهیم پایه ای معادلات دیفرانسیل جزئی، روش های عددی کلاسیک، یادگیری عمیق، مشتق گیری خودکار و بررسی تاریخی پژوهش های پیشین در یادگیری ماشین آگاه از فیزیک.
- **فصل سوم (شبکه های عصبی آگاه از فیزیک):** تشریح ریاضی و ساختاری چارچوب شبکه های آگاه از فیزیک، فرمول بندی مدل های زمان پیوسته و زمان گسسته، نحوه حل مسائل مستقیم و معکوس و نکات عملی در آموزش شبکه.
- **فصل چهارم (پیاده سازی و ارزیابی نتایج):** ارائه جزئیات پیاده سازی حل گر مرجع طیفی، تحلیل نتایج حل مستقیم معادله برگرز، نتایج شناسایی ضرایب در مسئله معکوس و بحث و مقایسه جامع یافته ها.
- **فصل پنجم (نتیجه گیری و پیشنهادها):** جمع بندی دستاوردهای پژوهش، تبیین محدودیت های موجود و ارائه چشم اندازها و پیشنهادهایی برای پژوهش های آتی.

فصل ۲

مبانی نظری و پیشینه پژوهش

هدف این فصل، تبیین و تشریح مفاهیم پایه‌ای است که چارچوب شبکه‌های عصبی آگاه از فیزیک بر مبنای آن‌ها شکل گرفته است. در ابتدا، تعاریف و ویژگی‌های کلی معادلات دیفرانسیل جزئی به همراه مثال‌های کلاسیک ارائه می‌گردد و سپس روش‌های عددی کلاسیک حل این معادلات به اختصار بررسی می‌شوند. در ادامه، ساختار شبکه‌های عصبی مصنوعی، الگوریتم‌های آموزش و بهینه‌سازی و تکنیک مشتق‌گیری خودکار تشریح می‌گردند. در بخش پایانی فصل، پیشینه پژوهش‌های پیشین در حوزه ترکیب یادگیری ماشین با اصول فیزیکی مرور می‌شود تا جایگاه پژوهش حاضر مشخص گردد.

۱-۲ معادلات دیفرانسیل جزئی

معادله دیفرانسیل جزئی، رابطه‌ای ریاضی میان یک تابع مجهول چندمتغیره و مشتقات جزئی آن نسبت به متغیرهای مستقل است. صورت عمومی یک معادله دیفرانسیل جزئی برای تابع مجهول $u(x)$ به فرم زیر بیان می‌شود:

$$F(x, u, Du, D^2u, \dots, D^k u) = 0, \quad (1-2)$$

که در آن $x = (x_1, \dots, x_d, t) \in \mathbb{R}^{d+1}$ بردار متغیرهای مستقل مکانی و زمانی، Du نمایانگر گرادیان یا مجموعه مشتقات جزئی مرتبه اول، D^2u ماتریس هشین یا مشتقات جزئی مرتبه دوم و $D^k u$ مشتقات جزئی مرتبه k ام هستند. بالاترین مرتبه مشتق موجود در رابطه را مرتبه معادله می‌نامند. چنانچه عملگر F نسبت به متغیر u و کلیه مشتقات

آن خطی باشد، معادله خطی و در غیر این صورت، غیرخطی^۱ نامیده می‌شود [۱۷]. برای تعیین جواب یکتا و فیزیکی از میان بی‌شمار جواب‌های عمومی معادله، مسئله باید همراه با شرایط مرزی^۲ و شرایط اولیه^۳ فرمول‌بندی شود. شرط اولیه حالت سیستم را در لحظه آغازین $t = 0$ مشخص نموده و شرایط مرزی نحوه برهم‌کنش سیستم با مرزهای دامنه مکانی $\partial\Omega$ را تعیین می‌کنند. بر اساس تعریف هادامارد، مسئله‌ای که دارای جواب یکتا باشد و جواب آن نسبت به داده‌های اولیه و مرزی پیوسته و پایدار بماند، یک مسئله خوش‌وضع^۴ نامیده می‌شود. در محاسبات علمی، برخی معادلات به دلیل تبیین پدیده‌های بنیادین، به عنوان مسائل معیار شناخته می‌شوند:

معادله گرما. این معادله ساده‌ترین و بنیادی‌ترین نمونه از معادلات سهموی است که انتقال و پخش دما را توصیف می‌کند. فرم یک‌بعدی آن عبارت است از:

$$u_t = \alpha u_{xx}, \quad (۲-۲)$$

که در آن $u(t, x)$ بیانگر توزیع دما در زمان t و مکان x ، و $\alpha > 0$ ضریب نفوذ یا پخش حرارتی است.

معادله برگرز. معادله برگرز^۵ نمونه‌ای بارز از معادلات دیفرانسیل جزئی غیرخطی است که می‌توان آن را مدل تقلیل‌یافته یک‌بعدی از معادلات ناویر-استوکس در دینامیک سیالات و مدل‌های جریان ترافیک دانست [۱۸]. صورت استاندارد معادله برگرز با لزجت به فرم زیر نوشته می‌شود:

$$u_t + uu_x - \nu u_{xx} = 0, \quad (۳-۲)$$

که در آن $\nu > 0$ ضریب لزجت سینماتیکی است. در این معادله، جمله غیرخطی uu_x نمایانگر انتقال (همرفت) است که تمایل به تیز کردن پروفیل موج و ایجاد ناپیوستگی دارد، در حالی که جمله νu_{xx} اثر پخششی داشته و به هموارسازی گرادیان‌ها می‌انجامد. تعامل این دو جمله، به‌ویژه در مقادیر بسیار کوچک ν ، منجر به پدیدار شدن جبهه‌های شیب‌دار تند و شبیه به موج شوک می‌شود. به همین علت، این معادله به معیاری چالش‌برانگیز و معتبر برای سنجش توانمندی روش‌های عددی تبدیل شده است و در

¹Nonlinear

²Boundary Conditions

³Initial Conditions

⁴Well-posed Problem

⁵Burgers Equation

این پژوهش نیز به عنوان مسئله آزمون مورد استفاده قرار گرفته است. **معادله غیرخطی شرودینگر.** این معادله در اپتیک غیرخطی، فیزیک پلاسما و مکانیک کوانتومی کاربرد گسترده‌ای دارد و در مقاله مرجع [۱] نیز به عنوان یکی از مسائل آزمون بررسی گردیده است.

۲-۲ روش‌های عددی کلاسیک

به جز در موارد خاص و با هندسه‌های بسیار متقارن، معادلات دیفرانسیل جزئی غیرخطی فاقد جواب تحلیلی بسته هستند؛ از این رو، روش‌های عددی نقش محوری را در حل عملی این مسائل بر عهده دارند. اساس کار روش‌های عددی، گسسته‌سازی دامنه پیوسته و تبدیل معادله دیفرانسیل به یک دستگاه معادلات جبری متناهی است [۱۶]. سه رویکرد اصلی در این زمینه عبارت‌اند از:

روش تفاضل‌های محدود. در روش تفاضل‌های محدود^۶، دامنه پیوسته با یک شبکه منظم از نقاط گسسته جایگزین شده و مشتقات جزئی با استفاده از بسط تیلور پیرامون نقاط همسایه تقریب زده می‌شوند. به عنوان نمونه، تقریب تفاضل مرکزی مرتبه دوم برای مشتق دوم مکانی به صورت زیر است:

$$u_{xx}(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2}, \quad (۴-۲)$$

که در آن Δx طول گام گسسته‌سازی مکانی است. مزیت عمده این روش، سادگی مفاهیم و سهولت پیاده‌سازی است؛ اما در مواجهه با دامنه‌های هندسی پیچیده و ناهموار، کارایی آن به شدت کاهش می‌یابد.

روش اجزای محدود. در روش اجزای محدود^۷، دامنه مکانی به زیردامنه‌های کوچکی به نام المان (نظیر مثلث‌ها یا چهارضلعی‌ها) تقسیم می‌گردد و جواب معادله بر حسب توابع پایه چندجمله‌ای موضعی بسط داده می‌شود. سپس با اعمال فرمول‌بندی ضعیف (روش وردشی یا گالرکین)، دستگاه معادلات جبری تشکیل می‌گردد. انعطاف‌پذیری فوق‌العاده در مدیریت هندسه‌های نامنظم و شرایط مرزی ترکیبی، این روش را به استاندارد مهندسی بدل ساخته است؛ هرچند پیچیدگی محاسباتی و هزینه الگوریتمی آن نسبتاً بالاست.

روش‌های طیفی. در روش‌های طیفی^۸، تابع جواب به صورت ترکیبی خطی از توابع

^۶Finite Difference Method

^۷Finite Element Method

^۸Spectral Methods

جدول ۲-۱: مقایسه ویژگی‌های روش‌های عددی کلاسیک در حل معادلات دیفرانسیل جزئی

روش محاسباتی	ایده بنیادین	مزیت اصلی	محدودیت اصلی
تفاضل‌های محدود	تقریب موضعی مشتقات روی شبکه ساخت‌یافته	سادگی الگوریتمی و پیاده‌سازی	ناتوانی در هندسه‌های نامنظم
اجزای محدود	فرمول‌بندی ضعیف روی المان‌های نامنظم	انعطاف‌پذیری هندسی بالا	هزینه بالای تولید شبکه و محاسبات
روش‌های طیفی	بسط جواب بر حسب توابع پایه سراسری	نرخ همگرایی (دقت) بسیار بالا	محدود به هندسه‌های ساده و شرایط هموار

پایه متعامد سراسری (نظیر چندجمله‌ای‌های چبیشف، لژاندر یا توابع مثلثاتی فوریه) تقریب زده می‌شود. مزیت برجسته روش‌های طیفی، نرخ همگرایی نمایی (موسوم به دقت طیفی) برای توابع با جواب‌های بی‌نهایت‌بار هموار است [۲۰]. در بخش تجربی این پایان‌نامه، از یک حل‌گر طیفی فوریه برای تولید داده‌های مرجع دقیق معادله برگرز استفاده شده است.

جدول ۲-۱ مقایسه‌ای اجمالی از ویژگی‌های این سه دسته روش را نشان می‌دهد. تمامی روش‌های کلاسیک در دو ویژگی اشتراک دارند: وابستگی ساختاری به تولید شبکه محاسباتی و ضرورت حل مجدد کل مسئله در صورت تغییر شرایط اولیه، شرایط مرزی یا پارامترهای سیستم. همچنین با افزایش ابعاد مسئله ($d \geq 3$)، پدیده نفرین ابعاد منجر به رشد نمایی تعداد درجات آزادی و هزینه‌های حافظه و محاسبات می‌گردد.

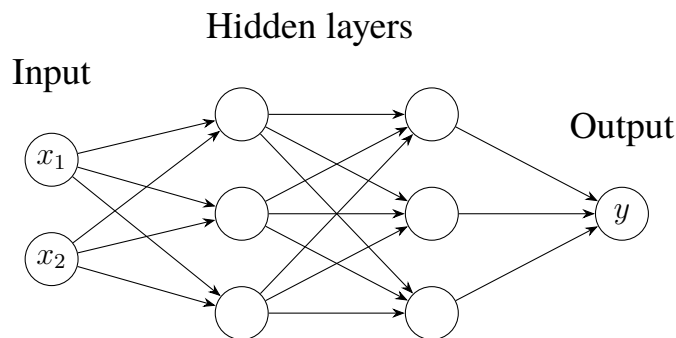
۳-۲ شبکه‌های عصبی مصنوعی

شبکه عصبی مصنوعی^۹ مدلی ریاضی و محاسباتی است که از پردازش اطلاعات در شبکه‌های بیولوژیکی الهام گرفته شده است. واحد بنیادی پردازش در این ساختارها، نورون مصنوعی است. هر نورون حاصل جمع وزن‌دار ورودی‌ها را با یک مقدار بایاس جمع نموده و آن را از یک تابع غیرخطی به نام تابع فعال‌ساز^{۱۰} عبور می‌دهد:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right), \quad (۵-۲)$$

^۹Artificial Neural Network (ANN)

^{۱۰}Activation Function



شکل ۲-۱: ساختار شماتیک یک شبکه عصبی پرسپترون چندلایه با دو متغیر ورودی، دو لایه پنهان و یک خروجی اسکالر

جدول ۲-۲: توابع فعال‌ساز متداول در شبکه‌های عصبی عمیق

عنوان تابع	فرمول ریاضی	ویژگی رفتاری
سیگموئید (Sigmoid)	$\sigma(z) = \frac{1}{1 + e^{-z}}$	بازه خروجی (0, 1)، هموار و مشتق‌پذیر
تانژانت هیپربولیک (Tanh)	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	بازه خروجی (-1, 1)، هموار، متقارن حول مبدأ
واحد خطی یکسوشده (ReLU)	$\text{ReLU}(z) = \max(0, z)$	محاسبه سریع، نامشتق‌پذیر در صفر، مشتق دوم صفر

که در آن x_i ورودی‌ها، w_i وزن‌های سیناپسی، b پارامتر بایاس و $\sigma(\cdot)$ تابع فعال‌ساز است. چیدمان لایه‌ای این واحدها، ساختار پرسپترون چندلایه^{۱۱} را پدید می‌آورد. تبدیل برداری در لایه l ام به صورت رابطه بازگشتی زیر تعریف می‌شود:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}), \quad (۲-۶)$$

که در آن $\mathbf{h}^{(0)} = \mathbf{x}$ بردار ورودی، $\mathbf{W}^{(l)}$ ماتریس وزن‌ها و $\mathbf{b}^{(l)}$ بردار بایاس لایه l ام است. ساختار گرافیکی یک پرسپترون چندلایه در شکل ۲-۱ به تصویر کشیده شده است. انتخاب تابع فعال‌ساز مناسب نقشی تعیین‌کننده در رفتار و قابلیت مشتق‌پذیری شبکه دارد. جدول ۲-۲ توابع فعال‌ساز متداول را مقایسه می‌نماید. در شبکه‌های عصبی آگاه از فیزیک، به کارگیری توابع بی‌نهایت‌بار مشتق‌پذیر و هموار نظیر تانژانت هیپربولیک ($\tanh$) امری ضروری است؛ زیرا تشکیل جملات دیفرانسیلی نیازمند محاسبه مشتقات مرتبه دوم یا بالاتر خروجی نسبت به ورودی‌هاست، در حالی که توابعی نظیر ReLU دارای مشتق دوم همه جا صفر هستند.

پایه‌های نظری توانمندی شبکه‌های عصبی در تقریب توابع با «قضیه تقریب

^{۱۱}Multi-Layer Perceptron (MLP)

عمومی» استوار شده است. بر اساس قضیه اثبات شده توسط سیپنکو [۹]:

قضیه ۱-۲ (تقریب عمومی سیپنکو). فرض کنید $\sigma(\cdot)$ یک تابع پیوسته سیگموئیدی (غیرخطی و کراندار) باشد. در این صورت، هر تابع پیوسته $f \in C(K)$ تعریف شده روی یک مجموعه فشرده $K \subset \mathbb{R}^n$ را می‌توان با یک شبکه عصبی پیش‌خور تک‌لایه پنهان با دقت دلخواه $\epsilon > 0$ تقریب زد.

هرچند این قضیه کفایت تئوریک شبکه‌های تک‌لایه با پهنای نامتناهی را اثبات می‌کند، اما در عمل شبکه‌های عمیق‌تر (با لایه‌های متوالی) با تعداد کل پارامترهای بسیار کمتر، قابلیت یادگیری توابع پیچیده فیزیکی را فراهم می‌سازند.

۴-۲ آموزش شبکه‌های عصبی عمیق

فرایند آموزش شبکه عصبی به معنای یافتن مجموعه پارامترهای بهینه $\theta^* = \{W, b\}$ است به‌گونه‌ای که یک تابع هدف مشخص کمینه گردد. در مسائل رگرسیون پیوسته، معیار میانگین مربعات خطا^{۱۲} پرکاربردترین تابع هزینه است:

$$\text{MSE}(\theta) = \frac{1}{N} \sum_{i=1}^N (u_{\theta}(\mathbf{x}_i) - u_i)^2, \quad (7-2)$$

که در آن $u_{\theta}(\mathbf{x}_i)$ پیش‌بینی مدل برای ورودی $\mathbf{x}_i$ و مقدار هدف است. کمینه‌سازی تابع هزینه با الگوریتم‌های گرادیان کاهشی انجام می‌شود:

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} \mathcal{L}(\theta_k), \quad (8-2)$$

که در آن $\eta > 0$ نرخ یادگیری^{۱۳} و $\nabla_{\theta} \mathcal{L}$ گرادیان تابع هزینه نسبت به بردار پارامترهاست. محاسبه کارآمد این گرادیان‌ها توسط الگوریتم پس‌انتشار خطا^{۱۴} صورت می‌گیرد که مبتنی بر قاعده زنجیره‌ای روی گراف محاسباتی است [۱۰].

در بهینه‌سازی شبکه‌های آگاه از فیزیک، ترکیب دو بهینه‌ساز زیر راهبردی استاندارد به شمار می‌رود:

بهینه‌ساز آدام (Adam). این بهینه‌ساز با محاسبه گشتاور اول و دوم متحرک

¹²Mean Squared Error (MSE)

¹³Learning Rate

¹⁴Backpropagation

گرادیان‌ها، نرخ یادگیری تطبیقی را برای هر پارامتر اعمال می‌کند [۱۲]:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1 - \beta_1) g_k, \\ v_k &= \beta_2 v_{k-1} + (1 - \beta_2) g_k^2, \\ \hat{m}_k &= \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}, \\ \theta_{k+1} &= \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}, \end{aligned} \quad (9-2)$$

که در آن $g_k = \nabla_{\theta} \mathcal{L}(\theta_k)$ ، $\beta_1 = 0.9$ ، $\beta_2 = 0.999$ و $\epsilon = 10^{-8}$ مقادیر استاندارد هستند. عبارات $\hat{m}_k$ و $\hat{v}_k$ اصلاح‌کننده بایاس اولیه در گام‌های آغازین بهینه‌سازی می‌باشند. **بهینه‌ساز شبه‌نیوتنی ال-بی‌اف‌جی‌اس (L-BFGS)**. این روش شبه‌نیوتنی با تقریب ماتریس معکوس هشین بر پایه تغییرات گرادیان در گام‌های پیشین، همگرایی مرتبه دو و فوق‌خطی در نزدیکی نقاط کمینه ایجاد می‌کند [۱۳]. در آموزش شبکه‌های آگاه از فیزیک، معمولاً ابتدا چند هزار گام با آدام اجرا می‌شود تا بهینه‌سازی از نواحی ناملازم عبور کند، و سپس برای رسیدن به دقت‌های بالای مهندسی، آموزش با الگوریتم L-BFGS تکمیل می‌گردد.

علاوه بر بهینه‌ساز، مقداردهی اولیه وزن‌ها نقشی حیاتی در همگرایی دارد. در این پژوهش از روش مقداردهی گزایه^{۱۵} استفاده شده است که واریانس وزن‌های هر لایه را متناسب با تعداد نورون‌های ورودی و خروجی آن تنظیم می‌نماید [۱۴].

۵-۲ مشتق‌گیری خودکار

مشتق‌گیری خودکار^{۱۶} مجموعه‌ای از تکنیک‌های محاسباتی برای ارزیابی مشتق توابع پیاده‌سازی شده در کدهای برنامه‌نویسی است [۱۵]. مشتق‌گیری خودکار از دو رویکرد کلاسیک دیگر متمایز است: بر خلاف مشتق‌گیری عددی (نظیر تفاضل محدود)، فاقد خطای برش و ناپایداری ناشی از گردکردن است و تا دقت ممیز شناور ماشین دقیق عمل می‌کند؛ و بر خلاف مشتق‌گیری نمادین، با عبارات جبری پیچیده روبه‌رو نمی‌شود و از پدیده رشد نمایی طول عبارات جبری مبرا است.

دو حالت اصلی در مشتق‌گیری خودکار عبارت‌اند از:

۱. **حالت مستقیم (Forward Mode):** مشتقات هم‌زمان با اجرای مستقیم برنامه و با محاسبه مشتقات جهت‌دار ارزیابی می‌شوند. این حالت برای توابعی با ابعاد

^{۱۵}Xavier Initialization

^{۱۶}Automatic Differentiation

ورودی اندک و خروجی زیاد کارآمد است.

۲. **حالت معکوس (Reverse Mode):** ابتدا اجرای مستقیم صورت گرفته و مقادیر متغیرهای میانی ذخیره می‌شوند؛ سپس با یک پیمایش بازگشتی از خروجی به سمت ورودی‌ها، گرادیان کامل نسبت به تمامی ورودی‌ها محاسبه می‌گردد. این حالت زیربنای الگوریتم پس‌انتشار است.

در چارچوب شبکه‌های آگاه از فیزیک، مشتق‌گیری خودکار کاربردی دوگانه دارد: نخست، محاسبه گرادیان تابع هزینه نسبت به وزن‌های شبکه جهت به‌روزرسانی پارامترها؛ و دوم (که مشخصه اختصاصی این چارچوب است)، محاسبه مشتقات جزئی متغیر وابسته نسبت به متغیرهای مستقل مکانی و زمانی (u_t, u_x, u_{xx}) برای ارزیابی مستقیم باقیمانده معادلات دیفرانسیل. کتابخانه‌های مدرن یادگیری عمیق نظیر جکس [۲۴] و تنسورفلو [۲۳] این قابلیت را با کامپایل کارآمد روی سخت‌افزارهای پردازشی فراهم می‌سازند.

۶-۲ پیشینه پژوهش

به‌کارگیری شبکه‌های عصبی در حل معادلات دیفرانسیل ریشه در پژوهش‌های دهه ۱۹۹۰ میلادی دارد. لاگاریس و همکارانش در سال ۱۹۹۸ نشان دادند که می‌توان جواب معادلات دیفرانسیل معمولی و جزئی را با پرسپترون‌های چندلایه فرمول‌بندی کرد به‌طوری که شرایط مرزی به‌صورت ساختاری ارضا شده و باقیمانده معادله روی نقاط آزمون کمینه گردد [۲]. با این حال، محدودیت توان پردازشی و عدم وجود کتابخانه‌های مشتق‌گیری خودکار در آن برهه، توسعه این روش‌ها را متوقف ساخت.

با پیشرفت علوم داده، رویکردهای یادگیری فیزیک‌پایه مجدداً کانون توجه قرار گرفتند. در این راستا، مدل‌سازی مبتنی بر فرایندهای گاوسی^{۱۷} برای استنتاج عملگرهای خطی [۳] و معادلات غیرخطی از طریق خطی‌سازی موضعی [۴] پیشنهاد گردید. این روش‌ها توانایی ارائه برآورد عدم قطعیت را دارا بودند، اما به دلیل فرضیات گاوسی، قابلیت تعمیم آن‌ها به مسائل به‌شدت غیرخطی با محدودیت روبه‌رو بود.

در رویکردی موازی، کارپاتنه و همکارانش چارچوب «علم داده هدایت‌شده با نظریه»^{۱۸} را برای ادغام اصول علمی در مدل‌های یادگیری ماشین معرفی نمودند [۵]. همچنین روش‌های شناسایی سیستم‌های دینامیکی بر پایه تنک‌سازی نظیر SINDy توسط برانتون و همکاران توسعه یافت [۷]؛ این روش‌ها اگرچه در کشف معادلات

¹⁷Gaussian Process

¹⁸Theory-Guided Data Science

حاکم موفق بودند، اما به پایگاه داده‌ای جامع از توابع کاندید و مشتق‌گیری عددی روی داده‌های نویزی وابسته بودند.

انتشار مقاله رئیسی، پردیکاریس و کارنیاداکیس در سال ۲۰۱۹ تحولی یکپارچه در این حوزه به وجود آورد [۱]. آن‌ها با ترکیب شبکه‌های عمیق و مشتق‌گیری خودکار، ساختاری عام برای حل مسائل مستقیم و معکوس ارائه دادند که نیازی به تولید شبکه محاسباتی، خطی‌سازی موضعی یا کتابخانه جملات کاندید ندارد. عمومیت این چارچوب و کارایی آن در مسائل متنوع دینامیک سیالات و فیزیک غیرخطی، موج گسترده‌ای از تحقیقات نظری و کاربردی را در قالب مقالات مروری جامع نظیر کومو و همکاران [۸] به همراه داشته است. پژوهش حاضر بر پایه همین چارچوب شکل گرفته و پیاده‌سازی و ارزیابی تحلیلی آن را در هر دو دسته مسائل مستقیم و معکوس دنبال می‌کند.

۷-۲ جمع‌بندی فصل

در این فصل، مبانی نظری مورد نیاز برای فهم و تحلیل شبکه‌های عصبی آگاه از فیزیک مرور شد: تعاریف و معادلات دیفرانسیل جزئی غیرخطی، روش‌های عددی کلاسیک و چالش‌های محاسباتی آن‌ها، مبانی ساختار و بهینه‌سازی شبکه‌های عصبی عمیق، مبانی مشتق‌گیری خودکار و تاریخچه تحولات یادگیری ماشین فیزیک‌پایه. در فصل بعد، فرمول‌بندی ریاضی و الگوریتمی شبکه‌های آگاه از فیزیک به تفصیل تبیین خواهد شد.

فصل ۳

شبکه‌های عصبی آگاه از فیزیک

در این فصل، ساختار ریاضی و الگوریتمی چارچوب «شبکه‌های عصبی آگاه از فیزیک» (PINN) که متدولوژی اصلی این پژوهش را تشکیل می‌دهد، به تفصیل تشریح می‌گردد. ابتدا صورت‌بندی کلی مسئله و ایده محوری روش تبیین شده و سپس ساختار مدل‌های زمان‌پیوسته و زمان‌گسسته در مسائل مستقیم معرفی می‌شوند. در ادامه، فرمول‌بندی مسئله معکوس برای کشف پارامترهای مجهول معادلات ارائه می‌گردد و در پایان، ملاحظات تجربی و عملیاتی حاکم بر آموزش این مدل‌ها مورد بحث قرار می‌گیرد [۱].

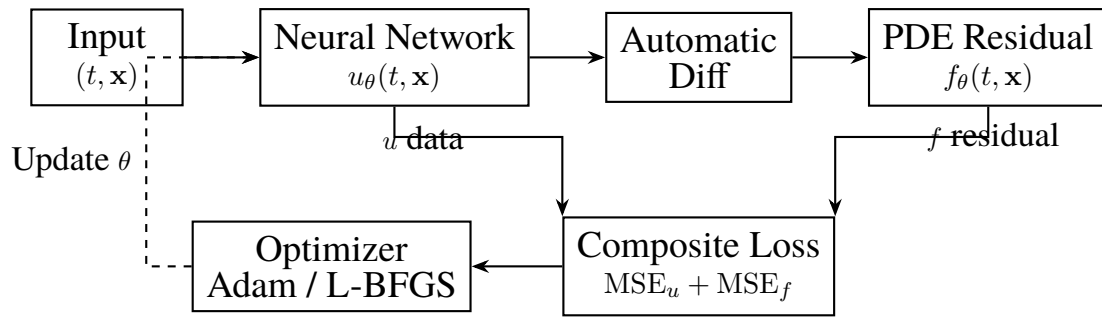
۱-۳ صورت‌بندی کلی مسئله و ایده محوری

معادلات دیفرانسیل جزئی غیرخطی و وابسته به زمان با ساختار پارامتری عام زیر را در نظر بگیرید:

$$u_t + \mathcal{N}[u; \lambda] = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, T], \quad (1-3)$$

که در آن $u(t, \mathbf{x})$ تابع مجهول حالت سیستم، $\mathcal{N}[\cdot; \lambda]$ عملگر دیفرانسیلی غیرخطی وابسته به بردار پارامترهای فیزیکی $\lambda \in \mathbb{R}^p$ و $\Omega \subset \mathbb{R}^d$ دامنه مکانی پیوسته با مرز $\partial\Omega$ است. این ساختار عمومی، طیف وسیعی از قوانین بقا، فرایندهای انتشار، معادلات ناویه‌استوکس و مدل‌های برگرز را شامل می‌شود. به عنوان نمونه، در معادله برگرز (۲-۳)، عملگر دیفرانسیلی به صورت $\mathcal{N}[u; \lambda] = \lambda_1 u u_x - \lambda_2 u_{xx}$ با بردار پارامتر $\lambda = (\lambda_1, \lambda_2)$ تعریف می‌گردد.

با فرض در اختیار داشتن مجموعه‌ای از داده‌های مشاهده‌شده (که ممکن است



شکل ۳-۱: نمودار بلوکی چارچوب شبکه عصبی آگاه از فیزیک: تعامل شبکه پاسخ، مشتق‌گیری خودکار، شبکه باقیمانده و حلقه بهینه‌سازی

محدود، نامنظم یا آلوده به نویز باشند)، دو مسئله بنیادین قابل طرح است:

۱. **مسئله مستقیم (Forward Problem):** با فرض معلوم بودن کامل بردار

پارامترها λ و شرایط مرزی و اولیه، هدف محاسبه و بازسازی میدان پاسخ $u(t, x)$ در سراسر دامنه زمانی-مکانی است.

۲. **مسئله معکوس (Inverse Problem):** در شرایطی که بردار پارامترهای λ

مجهول بوده و مقادیری از $u(t, x)$ در نقاط پراکنده‌ای از دامنه مشاهده شده باشد، هدف تخمین و شناسایی دقیق پارامترهای فیزیکی λ هم‌زمان با برازش میدان پاسخ است.

ایده محوری در شبکه‌های آگاه از فیزیک بر پایه تقریب تابع جواب $u(t, x)$ توسط یک شبکه عصبی عمیق $u_\theta(t, x)$ با بردار پارامترهای وزن و بایاس θ بنا شده است. بر این اساس، عملگر باقیمانده دیفرانسیلی فیزیک $f(t, x)$ به صورت زیر تعریف می‌گردد:

$$f(t, x) := u_t + \mathcal{N}[u; \lambda], \quad (۲-۳)$$

که در آن $u = u_\theta(t, x)$ خروجی شبکه عصبی است. ویژگی برجسته این چارچوب آن است که کلیه مشتقات جزئی ظاهرشده در f (نظیر u_t و مشتقات مکانی) با استفاده از مشتق‌گیری خودکار نسبت به ورودی‌های گراف محاسباتی ارزیابی می‌شوند. در نتیجه، $f(t, x)$ خود نمایانگر یک شبکه عصبی با پارامترهای مشترک θ است که اصطلاحاً «شبکه آگاه از فیزیک» نامیده می‌شود.

در صورتی که شبکه u_θ به جواب دقیق معادله میل کند، مقدار باقیمانده در تمامی نقاط دامنه به سمت صفر میل خواهد کرد ($f \rightarrow 0$). بنابراین، فرایند آموزش از طریق یک تابع هزینه ترکیبی که هم‌زمان خطای برازش داده‌ها و خطای عدم ارضای معادله دیفرانسیل را کمینه می‌سازد، هدایت می‌شود. شکل ۳-۱ تا ۳-۳ نمای شماتیک این ساختار یکپارچه را نشان می‌دهد.

۲-۳ مدل‌های زمان‌پیوسته برای مسئله مستقیم

در مدل‌های زمان‌پیوسته، متغیرهای فضا و زمان به صورت هم‌زمان به عنوان ورودی‌های پیوسته به شبکه عصبی تغذیه می‌شوند: $(t, \mathbf{x}) \mapsto u_\theta(t, \mathbf{x})$. پارامترهای مجهول شبکه θ با کمینه‌سازی تابع هزینه زیر در طول فرایند آموزش بهینه‌سازی می‌گردند:

$$\mathcal{L}(\theta) = \text{MSE}_u(\theta) + \text{MSE}_f(\theta), \quad (۳-۳)$$

که در آن:

$$\begin{aligned} \text{MSE}_u(\theta) &= \frac{1}{N_u} \sum_{i=1}^{N_u} |u_\theta(t_u^i, \mathbf{x}_u^i) - u^i|^2, \\ \text{MSE}_f(\theta) &= \frac{1}{N_f} \sum_{i=1}^{N_f} |f_\theta(t_f^i, \mathbf{x}_f^i)|^2. \end{aligned} \quad (۴-۳)$$

در روابط فوق، مجموعه $\{t_u^i, \mathbf{x}_u^i, u^i\}_{i=1}^{N_u}$ شامل نقاط داده روی شرایط مرزی و اولیه است که مقادیر دقیق متغیر حالت در آن‌ها مشخص است. مجموعه $\{t_f^i, \mathbf{x}_f^i\}_{i=1}^{N_f}$ نمایانگر «نقاط هم‌مکانی»^۱ است؛ نقاطی در درون دامنه پیوسته زمانی-مکانی که هیچ اندازه‌گیری تجربی از مقدار u در آن‌ها وجود ندارد و صرفاً ارضای ساختار معادله دیفرانسیل ($f \approx 0$) بر آن‌ها اعمال می‌گردد.

جمله MSE_f در حقیقت نقش یک منظم‌ساز فیزیکی قدرتمند را ایفا می‌کند که فضای فرضیات شبکه عصبی را به توابعی محدود می‌سازد که ساختار فیزیکی حاکم را برآورده کنند. در مسئله مستقیم معادله برگرز با لزجت ν ، باقیمانده فیزیکی صریح به صورت زیر تشکیل می‌گردد:

$$f := u_t + uu_x - \nu u_{xx}. \quad (۵-۳)$$

این ساختار به مدل اجازه می‌دهد با تعداد بسیار اندکی از داده‌های مرزی-اولیه، میدان پیوسته پاسخ را در سراسر دامنه بازسازی نماید.

^۱Collocation Points

۳-۳ مدل‌های زمان‌گسسته

در مسائلی که نیازمند پیشروی در بازه‌های زمانی بسیار طولانی یا دقت‌های زمانی فوق‌العاده بالا هستند، مدل‌های زمان‌پیوسته ممکن است با افزایش تعداد نقاط هم‌مکانی و دشواری بهینه‌سازی روبه‌رو شوند. برای رفع این چالش، فرمول‌بندی زمان‌گسسته با تلفیق طرح‌های گام‌زنی زمانی ضمنی از خانواده روش‌های رانگه-کوتا^۲ با مرحله میانی پیشنهاد شده است.

با اعمال صورت کلی یک روش رانگه-کوتا^۳ی ضمنی به معادله دیفرانسیل (۱-۳)، روابط گام زمانی به صورت زیر نوشته می‌شوند:

$$\begin{aligned} u^{n+c_i} &= u^n - \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u^{n+c_j}], \quad i = 1, \dots, q, \\ u^{n+1} &= u^n - \Delta t \sum_{j=1}^q b_j \mathcal{N}[u^{n+c_j}], \end{aligned} \quad (۶-۳)$$

که در آن ضرایب جدول بوچر^۳ $\{a_{ij}, b_j, c_j\}$ طرح رانگه-کوتا هستند [۲۲]. در این مدل، خروجی شبکه عصبی شامل $q + 1$ مقدار است که متناظر با جواب در مراحل میانی $u^{n+c_1}, \dots, u^{n+c_q}$ و جواب گام بعدی u^{n+1} می‌باشد. تابع هزینه بر اساس تطابق این مقادیر با روابط (۶-۳) و داده‌های لحظات t^n و t^{n+1} شکل می‌گیرد.

مزیت عمده این رهیافت، پایداری ذاتی روش‌های ضمنی رانگه-کوتا (نظیر خانواده گاوس-لژاندر) است که امکان انتخاب گام‌های زمانی بزرگ (Δt) بدون نقض شرایط پایداری را فراهم می‌آورد. در این پژوهش، تمرکز پیاده‌سازی و ارزیابی تجربی بر مدل زمان‌پیوسته قرار دارد، اما تحلیل مبانی مدل‌های زمان‌گسسته برای درک جامع ابعاد چارچوب ضروری است.

۴-۳ مسئله معکوس و کشف پارامترهای فیزیکی

در مسائل معکوس یا کشف معادلات حاکم، علاوه بر میدان مجهول $u(t, \mathbf{x})$ بردار ضرایب فیزیکی λ نیز مجهول بوده و باید مستقیماً از روی مشاهدات تجربی استخراج گردد. در چارچوب شبکه‌های آگاه از فیزیک، این فرایند با تعریف λ به عنوان متغیرهای بهینه‌سازی اضافی در کنار وزن‌های شبکه صورت می‌پذیرد.

^۲Runge-Kutta Methods

^۳Butcher Tableau

برای نمونه، در مسئله کشف پارامترهای معادله برگرز، عملگر باقیمانده به شکل پارامتری زیر در نظر گرفته می‌شود:

$$f := u_t + \lambda_1 u u_x - \lambda_2 u_{xx}, \quad (7-3)$$

که در آن λ_1 و λ_2 پارامترهای اسکالر ناشناخته هستند. فرایند بهینه‌سازی تابع هزینه (۳-۳) هم‌زمان نسبت به $(\theta, \lambda_1, \lambda_2)$ انجام می‌پذیرد.

تفاوت بنیادین این رهیافت با روش‌های متداول تحلیل داده (نظیر رگرسیون نمادین یا مشتق‌گیری تفاضلی روی داده‌های نویزی) در آن است که مشتقات جزئی مورد نیاز از روی شبکه هموار u_θ با مشتق‌گیری خودکار ارزیابی می‌شوند. این امر باعث می‌شود که شبکه عصبی هم‌زمان نقش یک فیلتر هموارکننده نویز را ایفا کرده و پایداری محاسباتی بالایی در برابر خطاهای اندازه‌گیری فراهم آورد.

۳-۵ ملاحظات عملی در پیاده‌سازی و آموزش

تجارب محاسباتی و مبانی تجربی حاکی از آن است که دستیابی به همگرایی مطلوب در شبکه‌های آگاه از فیزیک مستلزم رعایت چند اصل کلیدی در طراحی ساختار و الگوریتم آموزش است:

معماری شبکه و توابع فعال‌ساز. در اغلب مسائل فیزیکی، پرسپترون‌های چندلایه با عمق ۴ تا ۸ لایه و ۲۰ تا ۵۰ نورون در هر لایه با تابع فعال‌ساز تانژانت هیپربولیک ($\tanh$) عملکرد مطلوبی نشان می‌دهند. به‌کارگیری لایه‌های افت تصادفی (Dropout) در این شبکه‌ها معمولاً غیرضروری است؛ زیرا قید فیزیکی باقیمانده به‌طور خودکار مانع بیش‌برازش می‌گردد.

راهبرد بهینه‌سازی ترکیبی. استفاده از الگوریتم دو مرحله‌ای شامل شروع با بهینه‌ساز آدام (جهت فرار از کمینه‌های محلی و کاهش سریع خطای اولیه) و سپس تغییر وضعیت به بهینه‌ساز شبه‌نیوتنی L-BFGS (جهت رسیدن به دقت‌های بالا و همگرایی مرتبه دو) پایدارترین استراتژی آموزشی است.

راهبرد نمونه‌برداری نقاط هم‌مکانی. توزیع و تراکم نقاط هم‌مکانی در دامنه حل بر پایداری مدل اثرگذار است. در کنار نمونه‌برداری یکنواخت تصادفی یا روش ابرمکعب لاتین^۴، افزایش چگالی نقاط هم‌مکانی در نواحی با تغییرات مکانی شدید (نظیر جبهه شوک در معادله برگرز) دقت تقریب را به میزان چشمگیری بهبود می‌بخشد.

⁴Latin Hypercube Sampling

جدول ۳-۱: مقایسه تطبیقی مدل‌های زمان‌پیوسته و زمان‌گسسته در چارچوب شبکه‌های آگاه از فیزیک

شاخص ساختاری	مدل زمان‌پیوسته	مدل زمان‌گسسته
ورودی شبکه	بردار پیوسته زمانی-مکانی (t, x)	متغیر مکانی x در یک مقطع زمانی
خروجی شبکه	مقدار اسکالر میدان $u(t, x)$	مقادیر مراحل میانی رانگه-کوتا u^{n+1} و u^n
داده‌های آموزشی	شرایط اولیه و مرزی در دامنه زمان	مقادیر حالت در دو لحظه گسسته t^{n+1} و t^n
مزیت شاخص	سادگی و یکپارچگی ساختار	پایداری در گام‌های زمانی بزرگ
وضعیت در این پژوهش	پیاده‌سازی و تحلیل کامل تجربی	تحلیل و بررسی نظری

جدول ۳-۱ مشخصات و تمایزات ساختاری مدل‌های زمان‌پیوسته و زمان‌گسسته را مقایسه می‌نماید.

۳-۶ جمع‌بندی فصل

در این فصل، ساختار ریاضی و متدولوژی شبکه‌های عصبی آگاه از فیزیک تشریح شد: مکانیزم تعریف شبکه باقیمانده بر پایه مشتق‌گیری خودکار، طراحی توابع هزینه ترکیبی فیزیک-داده، فرمول‌بندی مدل‌های زمان‌پیوسته و زمان‌گسسته برای مسائل مستقیم و ساختار استخراج ضرایب در مسائل معکوس. در فصل چهارم، پیاده‌سازی این چارچوب برای معادله برگرز یک‌بعدی و نتایج عددی حاصل ارائه خواهد شد.

فصل ۴

پیاده‌سازی و ارزیابی نتایج

در این فصل، جزئیات محاسباتی و نتایج تجربی حاصل از پیاده‌سازی چارچوب شبکه‌های عصبی آگاه از فیزیک ارائه می‌گردد. ابتدا بستر نرم‌افزاری و ابزارهای مورد استفاده معرفی شده و سپس حل‌گر مرجع طیفی توسعه‌یافته برای معادله غیرخطی برگرز تشریح و با آزمون‌های مستقل اعتبارسنجی می‌شود. در ادامه، نتایج حل مستقیم معادله با مدل زمان‌پیوسته و نتایج کشف پارامترها در مسئله معکوس در سناریوهای داده تمیز و نویزی گزارش و تحلیل می‌گردند. در پایان، یافته‌های تجربی با نتایج مقاله مرجع مقایسه و نقاط قوت و محدودیت‌های روش تبیین می‌شود.

۴-۱ بستر محاسباتی و ابزارهای پیاده‌سازی

کلیه الگوریتم‌های محاسباتی این پژوهش با زبان برنامه‌نویسی پایتون نسخه ۱۱.۳ پیاده‌سازی شده‌اند. برای پیاده‌سازی ساختار شبکه عصبی، کامپایل توابع و انجام مشتق‌گیری خودکار، کتابخانه جکس^۱ به کار گرفته شده است [۲۴]. جکس امکان ترکیب تبدیل‌های تابعی نظیر مشتق‌گیری خودکار (grad)، برداری‌سازی خودکار (vmap) و کامپایل در زمان اجرا (jit) را بر بستر کامپایلر XLA فراهم می‌سازد. انجام محاسبات عددی و ماتریسی با استفاده از کتابخانه‌های نامپای و سای‌پای، و رسم نمودارها با کتابخانه مت‌پلات‌لیب صورت پذیرفته است. الگوریتم بهینه‌سازی ترکیبی شامل پیاده‌سازی مستقیم بهینه‌ساز آدام و استفاده از رابط الگوریتم بهینه‌سازی مقید L-BFGS-B در کتابخانه سای‌پای است.

تمامی مراحل آموزش و آزمایش‌های عددی بر روی واحد پردازش مرکزی (CPU) و با دقت ممیز شناور ۶۴ بیتی (Float64) اجرا شده‌اند تا امکان بازتولید دقیق نتایج بر

^۱JAX

روی تجهیزات استاندارد دانشگاهی فراهم باشد. به منظور تضمین تکرارپذیری، بذری تولید اعداد تصادفی ثابت نگاه داشته شده و کد کامل برنامه به همراه راهنمای اجرا در پیوست الف و پوشه اختصاصی code ارائه گردیده است.

۲-۴ حل گر مرجع طیفی و اعتبارسنجی آن

جهت ارزیابی مستقل و دقیق عملکرد شبکه آگاه از فیزیک، یک حل گر عددی مرجع مستقل برای معادله برگرز با لزجت $\nu = 0.01/\pi$ توسعه داده شد. در این حل گر، گسسته سازی مکانی با روش طیفی فوریه بر روی ۱۰۲۴ مود با اعمال قاعده پالایش دو-سوم برای حذف خطای هم پوشانی^۲ صورت گرفته است [۲۰]. گام زنی زمانی با استفاده از روش تفاضل زمانی نمایی رانگه-کوتای مرتبه چهار (ETDRK4) با طول گام زمانی $\Delta t = 0.001$ انجام می پذیرد که برای معادلات دارای جملات پخش سفت، پایداری فوق العاده ای ارائه می دهد [۲۱]. ماتریس پاسخ حاصل بر روی یک شبکه زمانی-مکانی با ابعاد 101×1024 ذخیره شده و به عنوان معیار حل دقیق در ارزیابی ها استفاده شده است.

صحت عملکرد حل گر مرجع با دو آزمون مستقل اثبات گردید:

۱. **آزمون همگرایی عددی:** با نصف کردن گام زمانی ($\Delta t = 0.0005$)، بیشینه تغییر

نسبی جواب در مرتبه 6.2×10^{-9} و با دو برابر کردن تفکیک مکانی ($N_x = 2048$)، تغییر نسبی برابر 4.1×10^{-4} ثبت گردید که مؤید همگرایی مطلوب روش است.

۲. **آزمون مقایسه با جواب نیمه تحلیلی کول-هوپف:** جواب معادله با تبدیل تحلیلی

کول-هوپف [۱۸] از طریق انتگرال گیری عددی مستقیم با دقت بسیار بالا محاسبه

شد. بیشترین اختلاف در ۱۹ نقطه نمونه برداری در زمان نهایی $t = 1$ کمتر از

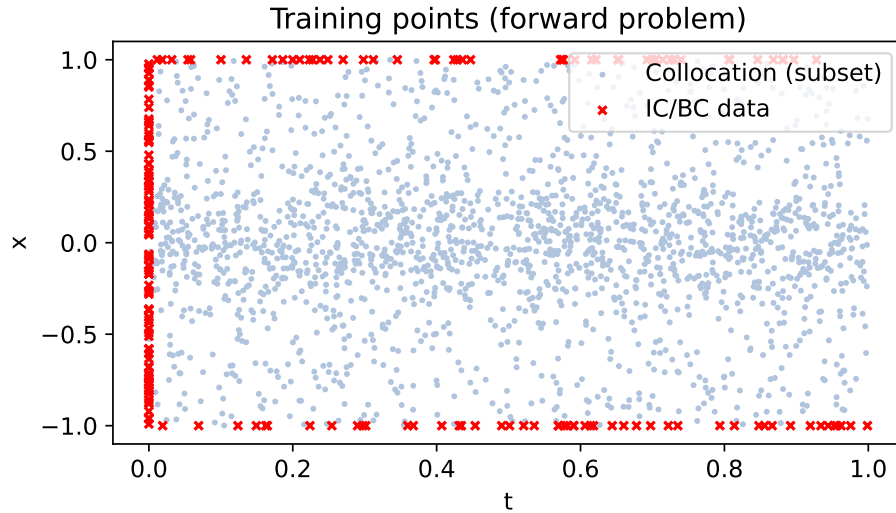
6×10^{-4} بوده و در نواحی دور از جبهه شوک به مرتبه 10^{-8} کاهش یافت. همچنین

مطابقت با بسط های تحلیلی سری فوریه [۱۹] نیز بررسی گردید.

این آزمون ها نشان می دهند که دقت حل مرجع چندین مرتبه بزرگی فراتر از دقت

هدف شبکه های عصبی بوده و مبنایی کاملاً قابل اعتماد برای ارزیابی خطاست.

²Aliasing Error



شکل ۴-۱: توزیع نقاط آموزشی در مسئله مستقیم معادله برگرز: نقاط داده اولیه و مرزی ($N_u = 200$) و نقاط هم‌مکانی باقیمانده ($N_f = 10000$)

۳-۴ حل مستقیم معادله برگرز

در مسئله مستقیم، هدف محاسبه جواب $u(t, x)$ در دامنه مکانی $x \in [-1, 1]$ و زمانی $t \in [0, 1]$ با شرط اولیه $u(0, x) = -\sin(\pi x)$ و شرایط مرزی همگن دیریکله $u(t, -1) = u(t, 1) = 0$ و ضرایب معلوم $\lambda_1 = 1$ و $\lambda_2 = 0.01/\pi$ است. ساختار شبکه عصبی مورد استفاده، پرسپترون چندلایه با ۴ لایه پنهان، ۵۰ نورون در هر لایه و تابع فعال‌ساز $\tanh$ است که در مجموع شامل ۷۸۵۱ پارامتر قابل آموزش می‌باشد.

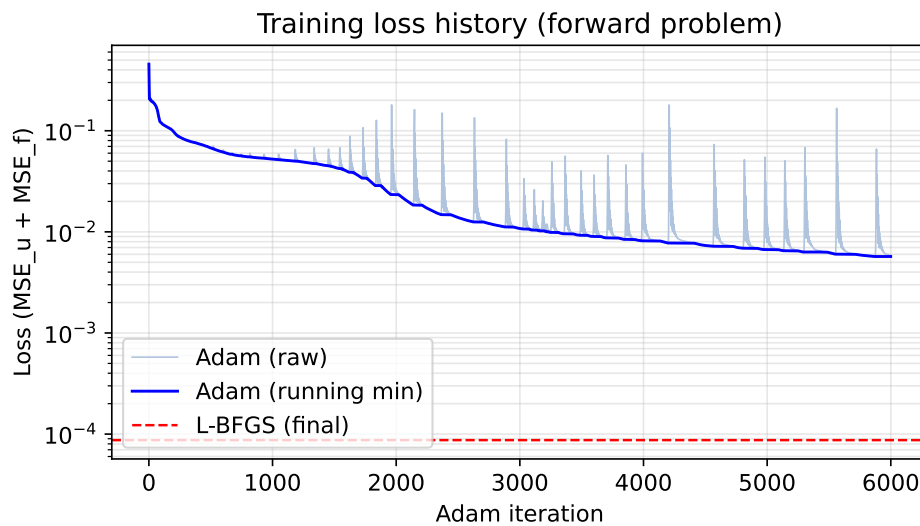
داده‌های آموزشی روی مرزها و شرط اولیه شامل ۲۰۰ نقطه ($N_u = 200$) شامل ۱۰۰ نقطه در $t = 0$ و ۱۰۰ نقطه روی مرزهای مکانی است. تعداد نقاط هم‌مکانی درون دامنه برابر ۱۰۰۰۰ نقطه ($N_f = 10000$) انتخاب شد که نیمی از آن‌ها به صورت تصادفی یکنواخت و نیمی دیگر با توزیع نرمال متمرکز حول خط $x = 0$ (ناحیه تشکیل جبهه شوک) نمونه‌برداری گردیدند. موقعیت نقاط آموزشی در شکل ۴-۱ و مشخصات ساختاری آموزش در جدول ۴-۱ خلاصه شده است.

فرایند آموزش طی دو مرحله متوالی اجرا گردید: ۶۰۰۰ گام با الگوریتم آدام با نرخ یادگیری 10^{-3} و سپس ادامه بهینه‌سازی با روش L-BFGS-B تا دستیابی به همگرایی. مقدار تابع هزینه نهایی در انتهای مرحله آدام برابر 5.85×10^{-3} و در پایان مرحله L-BFGS برابر 8.71×10^{-5} حاصل شد که در آن سهم مولفه خطای داده MSE_u برابر 2.35×10^{-5} و مولفه خطای باقیمانده فیزیک MSE_f برابر 6.36×10^{-5} بود. تاریخچه تغییرات تابع هزینه در شکل ۴-۲ رسم گردیده است.

ارزیابی شبکه آموزش‌دیده بر روی کل شبکه مرجع با ۱۰۳۴۲۴ نقطه (101×1024)

جدول ۴-۱: تنظیمات ساختاری و پارامترهای آموزش مدل زمان‌پیوسته در مسئله مستقیم

مشخصات انتخابی	مولفه ساختاری
۴ لایه پنهان، ۵۰ نورون در هر لایه، فعال‌ساز تانژانت هیپربولیک	معماری شبکه عصبی
۷۸۵۱ پارامتر وزن و بایاس	تعداد کل پارامترهای شبکه
۲۰۰ نقطه (۱۰۰ شرط اولیه و ۱۰۰ شرط مرزی)	نقاط داده اولیه و مرزی (N_u)
۱۰۰۰۰ نقطه (۵۰۰۰ یکنواخت و ۵۰۰۰ متمرکز در ناحیه شوک)	نقاط هم‌مکانی باقیمانده (N_f)
روش گزایه (گلوروت)	الگوریتم مقداردهی اولیه
۶۰۰۰ گام آدام ($\eta = 10^{-3}$) و سپس L-BFGS-B تا ۶۰۰۰ فراخوانی	راهبرد بهینه‌سازی
۶۴ بیتی (Float64)	دقت محاسباتی ممیز شناور



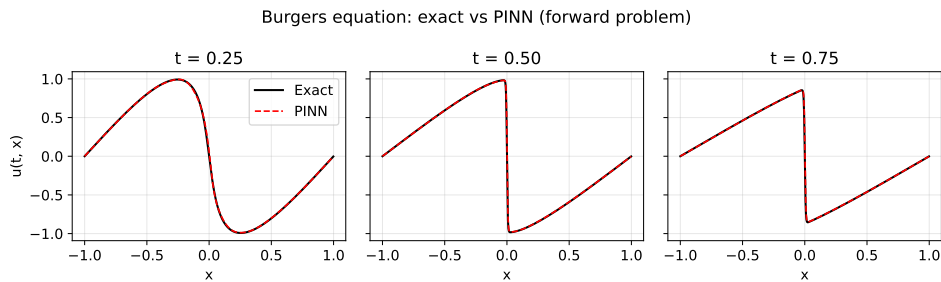
شکل ۴-۲: تاریخچه کاهش تابع هزینه در مسئله مستقیم: همگرایی مرحله اول با الگوریتم آدام و اصلاح نهایی با روش شبه‌نیوتنی L-BFGS

نشان‌دهنده دستیابی به خطای نسبی در نرم L_2 برابر 2.15×10^{-2} است:

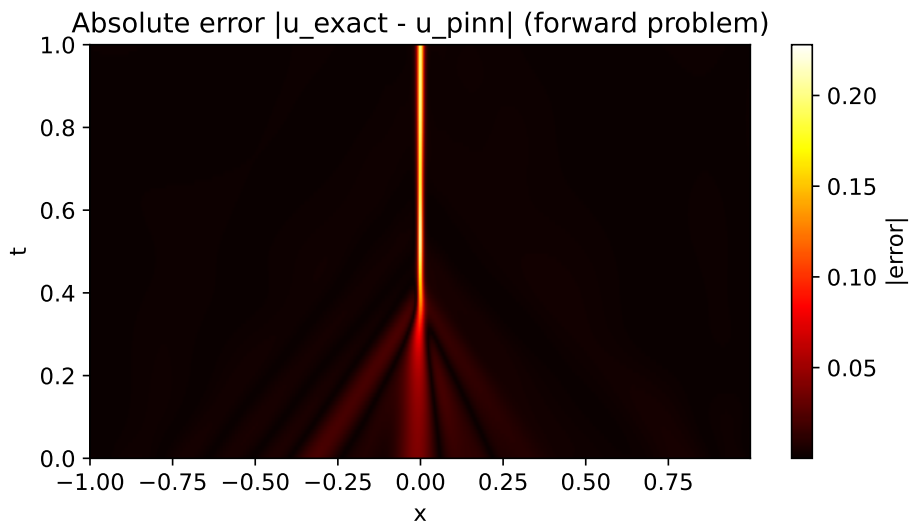
$$\text{Relative } L_2 \text{ Error} = \frac{\|u_{\text{pred}} - u_{\text{ref}}\|_2}{\|u_{\text{ref}}\|_2} = 2.15 \times 10^{-2}. \quad (1-4)$$

مقایسه پروفیل‌های زمانی پاسخ پیش‌بینی‌شده با حل مرجع در مقاطع $t = 0.25$ ، $t = 0.5$ و $t = 0.75$ در شکل ۴-۳ نمایش داده شده است. شکل ۴-۴ نقشه خطای قدرمطلق را در سراسر دامنه زمانی-مکانی نشان می‌دهد. این نمودار تبیین می‌کند که خطای تقریب در بخش اعظم دامنه ناچیز بوده و بیشینه آن در لایه باریک گرادیان تند شوک (حوالی $x = 0$ برای $t > 0.3$) متمرکز است.

جدول ۴-۲ مقایسه نتایج این پژوهش را با مقاله مرجع [۱] نشان می‌دهد. تفاوت



شکل ۴-۳: مقایسه پروفیل‌های زمانی حل دقیق مرجع و پیش‌بینی شبکه آگاه از فیزیک در سه لحظه $t = 0.25, 0.5, 0.75$



شکل ۴-۴: نقشه توزیع خطای قدرمطلق $|u_{ref} - u_{PINN}|$ در دامنه زمانی-مکانی در مسئله مستقیم

اندک در دقت به عواملی نظیر تفاوت در تعداد لایه‌ها و عمق شبکه (۴ لایه ۵۰ نرونی در برابر ۹ لایه ۲۰ نرونی)، تفاوت پلتفرم سخت‌افزاری پردازش و سقف تعداد تکرارهای بهینه‌سازی مربوط می‌شود.

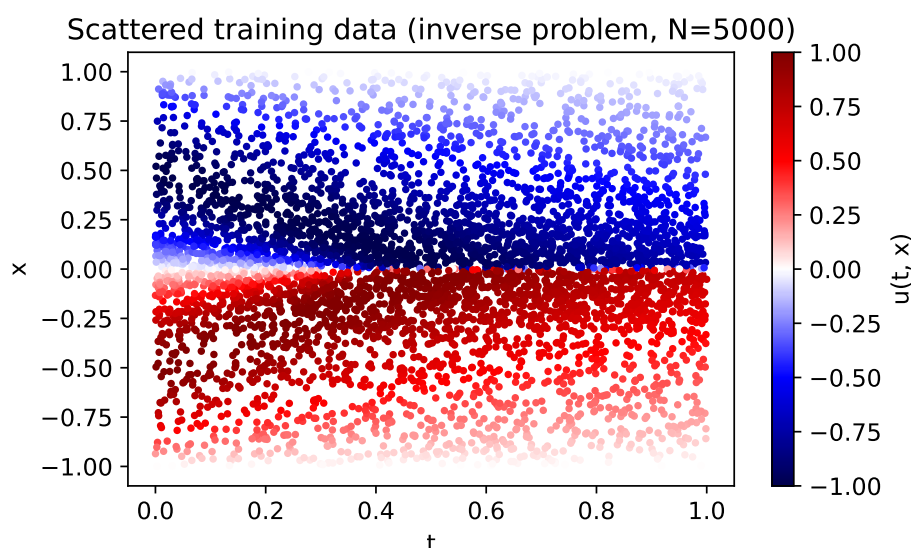
۴-۴ کشف پارامترهای نامعلوم در مسئله معکوس

در مسئله معکوس، فرض بر این است که ضرایب معادله برگرز در عملگر باقیمانده $f = u_t + \lambda_1 u u_x - \lambda_2 u_{xx}$ مجهول بوده و هدف، استخراج هم‌زمان پارامترهای λ_1 و λ_2 از روی مشاهدات تجربی پراکنده است. مجموعه داده‌های آموزشی شامل ۵۰۰۰ مشاهده پراکنده ($N = 5000$) در دامنه زمانی-مکانی است که نیمی به صورت یکنواخت و نیمی متمرکز حول $x = 0$ توزیع شده‌اند (شکل ۴-۵).

ساختار شبکه شامل ۵ لایه پنهان با ۳۰ نرون در هر لایه است. مقادیر اولیه هر دو ضریب برابر صفر ($\lambda_1 = 0, \lambda_2 = 0$) تعیین شد. بهینه‌سازی مطابق راهبرد ترکیبی ۶۰۰۰

جدول ۴-۲: مقایسه تطبیقی نتایج حل مستقیم با مقاله مرجع رئیسی و همکاران (۲۰۱۹)

منبع پژوهش	معماری شبکه	داده مرزی	نقاط هم مکانی	خطای نسبی L_2
رئیسی و همکاران [۱]	۹ لایه، ۲۰ نورون	۱۰۰	۱۰۰۰۰	6.7×10^{-4}
پژوهش حاضر	۴ لایه، ۵۰ نورون	۲۰۰	۱۰۰۰۰	2.15×10^{-2}



شکل ۴-۵: توزیع مکانی-زمانی ۵۰۰۰ نقطه مشاهده پراکنده در مسئله معکوس؛ طیف رنگی نمایانگر مقدار متغیر حالت $u(t, x)$ است

گام و آدام و ادامه با L-BFGS-B انجام گرفت.

مسئله معکوس در دو سناریوی آزمایشی بررسی شد:

۱. سناریوی داده تمیز (بدون نویز): داده‌ها مستقیماً از حل مرجع استخراج شدند.

۲. سناریوی داده نویزی: داده‌ها با نویز گاوسی با انحراف معیار برابر یک درصد انحراف

معیار سیگنال آلوده شدند $(u_{\text{noisy}} = u + 0.01 \times \text{std}(u) \times \mathcal{N}(0, 1))$.

جدول ۴-۳ مقادیر ضرایب شناسایی شده و درصد خطای نسبی آن‌ها را در مقایسه

با مقادیر واقعی $(\lambda_1^* = 1.0)$ و $(\lambda_2^* = 0.01/\pi \approx 0.003183)$ گزارش می‌نماید.

۴-۵ بحث و تحلیل نتایج

تحلیل نتایج عددی حاصل از پیاده‌سازی، نکات علمی و مهندسی متعددی را در خصوص رفتار شبکه‌های آگاه از فیزیک روشن می‌سازد:

جدول ۴-۳: نتایج تخمین پارامترهای مجهول معادله برگرز در مسئله معکوس

سناریوی داده	λ_1 برآوردی	خطای λ_1	λ_2 برآوردی	خطای λ_2
بدون نویز (تمیز)	0.912	8.8%	4.22×10^{-3}	32.6%
آلوده به ۱٪ نویز	0.923	7.7%	3.97×10^{-3}	24.9%
مقادیر واقعی هدف	1.000		$0.01/\pi \approx 0.003183$	

کارایی داده‌ای بالا. در مسئله مستقیم، تقریب میدان پیوسته پاسخ در بیش از یکصد هزار نقطه محاسباتی تنها بر مبنای ۲۰۰ نقطه داده اولیه و مرزی صورت پذیرفت. این نسبت نشان‌دهنده نقش برجسته قید باقیمانده فیزیکی در کاهش ابعاد فضای جست‌وجو و جلوگیری از بیش‌برازش است.

چالش لایه‌های با گرادیان تند. تمرکز عمده خطا در حوالی جبهه شوک نشان می‌دهد که پدیده سلبیت و تغییرات ناگهانی گرادیان، اصلی‌ترین گلوگاه دقتی شبکه‌های عصبی استاندارد در حل معادلات هذلولوی و شبه‌هذلولوی است. نمونه‌برداری غیریکنواخت و متمرکز در این ناحیه نقش تعیین‌کننده‌ای در کنترل خطای عددی ایفا نمود.

تحلیل حساسیت و خطای شناسایی ضریب پخش. خطای شناسایی ضریب انتقال (λ_1) در هر دو حالت کمتر از ۹ درصد بوده است، در حالی که خطای ضریب پخش (λ_2) در بازه ۲۵ تا ۳۳ درصد قرار دارد. این تفاوت ناشی از ماهیت فیزیکی مسئله است: با توجه به کوچکی مقدار $\lambda_2 \approx 0.00318$ ، اثر جمله پخشی $\lambda_2 u_{xx}$ در غالب نقاط دامنه در برابر جمله جابه‌جایی ناچیز است و صرفاً در لایه بسیار باریک شوک اثرگذار می‌شود؛ بنابراین گرادیان اطلاعاتی موجود در داده‌ها برای تنظیم λ_2 بسیار ضعیف‌تر از λ_1 است. **پایداری در برابر نویز و رفتار بهینه‌سازی.** نتایج جدول ۴-۳ نشان می‌دهد که افزودن نویز یک درصد ساختار تخمین پارامترها را دچار واگرایی نکرده و خطای شناسایی در همان مرتبه داده‌های تمیز باقی مانده است. تفاوت جزئی در خطای عددی میان دو سناریو، ناشی از نوسانات تصادفی بهینه‌سازی غیرمحدب و تفاوت در بذر تصادفی مقداردهی اولیه است و بیانگر پایداری رضایت‌بخش روش در برابر نویز به دلیل فیلترسازی طبیعی توسط شبکه عصبی هموار است.

مقایسه با روش‌های کلاسیک. در حالی که حل‌گرهای کلاسیک طیفی در تولید پاسخ برای یک مسئله معین با سرعت و دقت بسیار بالاتری عمل می‌کنند، چارچوب شبکه‌های آگاه از فیزیک در مسائلی که داده‌ها ناقص بوده، شرایط مرزی ناشناخته باشند، یا پارامترهای سیستم مجهول باشند، انعطاف‌پذیری بی‌نظیری از خود نشان می‌دهد. **محدودیت‌های پژوهش حاضر.** این پژوهش بر روی معادله برگرز یک‌بعدی و مدل

زمان پیوسته متمرکز بوده و بهینه سازی ابرپارامترها به دلیل محدودیت توان پردازشی در مقیاس محدودی انجام گرفته است. همچنین تحلیل عدم قطعیت پارامترهای استخراج شده بررسی نشده است.

۶-۴ جمع بندی فصل

در این فصل، پیاده سازی و ارزیابی محاسباتی مدل زمان پیوسته شبکه آگاه از فیزیک بر روی معادله برگرز ارائه شد. حل گر مرجع طیفی با گام زنی نمایی مرتبه چهار توسعه یافته و اعتبارسنجی شد. نتایج نشان داد که مدل زمان پیوسته با ۲۰۰ نقطه داده اولیه-مرزی به خطای نسبی 2.15×10^{-2} در مسئله مستقیم دست می یابد. در مسئله معکوس نیز ضرایب با دقت مناسبی استخراج شدند و پایداری روش در حضور نویز داده ها مورد تایید قرار گرفت.

فصل ۵

نتیجه‌گیری و پیشنهادها

در این فصل، ابتدا خلاصه‌ای جامع از فرایند پژوهش و اقدامات انجام‌شده ارائه می‌گردد. سپس دستاوردها و یافته‌های کلیدی پژوهش تبیین شده و در پایان، پیشنهادهایی هدفمند برای ادامه و گسترش این پژوهش در مطالعات آتی ارائه می‌شود.

۵-۱ خلاصه پژوهش

در این پایان‌نامه، چارچوب «شبکه‌های عصبی آگاه از فیزیک» (PINN) به‌عنوان رویکردی نوین و میان‌رشته‌ای برای حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی مورد بررسی نظری، پیاده‌سازی الگوریتمی و ارزیابی تجربی قرار گرفت. در بخش نظری، پس از مرور اصول معادلات دیفرانسیل جزئی، روش‌های عددی کلاسیک (تفاضل محدود، اجزای محدود و روش‌های طیفی) و محدودیت‌های آن‌ها در ابعاد بالا، مبانی یادگیری عمیق و سازوکار مشتق‌گیری خودکار تشریح شد. سپس ساختار ریاضی شبکه باقیمانده، نحوه تشکیل توابع هزینه ترکیبی و فرمول‌بندی مدل‌های زمان‌پیوسته و زمان‌گسسته تبیین گردید.

در بخش عملی و محاسباتی، مدل زمان‌پیوسته برای معادله غیرخطی برگرز با استفاده از زبان برنامه‌نویسی پایتون و کتابخانه جکس پیاده‌سازی شد. به‌منظور ارزیابی دقیق نتایج، یک حل‌گر مرجع طیفی مستقل بر پایه روش فوریه و گام‌زنی نمایی مرتبه چهار (ETDRK4) توسعه یافت و صحت عملکرد آن با تبدیل نیمه‌تحلیلی کول-هوپف اثبات گردید. آزمایش‌های عددی در دو بخش حل مستقیم با داده‌های مرزی محدود و کشف پارامترهای مجهول معادله در شرایط بدون نویز و همراه با نویز به اجرا درآمد و ویژگی‌های همگرایی و خطای روش مورد تحلیل قرار گرفت.

۲-۵ یافته‌ها و دستاوردهای کلیدی

مهم‌ترین یافته‌ها و دستاوردهای این پژوهش را می‌توان در محورهای زیر خلاصه نمود:

۱. **کارایی داده‌ای بالا در حل مسائل مستقیم:** مدل زمان‌پیوسته توانست با بهره‌گیری از تنها ۲۰۰ نقطه داده اولیه و مرزی، میدان پاسخ پیوسته معادله برگرز را در سراسر دامنه زمانی-مکانی بازسازی نموده و به خطای نسبی 2.15×10^{-2} در نرم L_2 نسبت به حل مرجع دست یابد. این امر مؤید آن است که افزودن باقیمانده فیزیکی به تابع هزینه، نیاز به پایگاه داده‌های حجیم را برطرف می‌سازد.
 ۲. **تمرکز مکانی خطا در لایه‌های با گرادیان تند:** توزیع مکانی خطای مطلق نشان داد که بیشینه خطا در لایه باریک پیرامون جبهه شوک متمرکز است. اعمال راهبرد نمونه‌برداری غیریکنواخت و متمرکز در این ناحیه، عاملی تعیین‌کننده در مهار خطا و بهبود دقت تقریب بود.
 ۳. **شناسایی موفق پارامترهای مجهول در مسئله معکوس:** ضرایب انتقال (λ_1) و پخش (λ_2) از روی ۵۰۰۰ مشاهده پراکنده استخراج شدند. ضریب انتقال با خطای کمتر از ۹ درصد در هر دو حالت شناسایی شد. خطای بیشتر در برآورد ضریب پخش (32.6%) ناشی از اثرگذاری موضعی و مقیاس کوچک این ضریب در مقایسه با جمله انتقال است.
 ۴. **پایداری ساختاری در برابر نویز اندازه‌گیری:** ارزیابی مدل در حضور یک درصد نویز نشان داد که ساختار پیوسته و هموار شبکه عصبی به‌صورت یک فیلتر نویز عمل کرده و خطای تخمین پارامترها را در محدوده قابل‌قبول حفظ می‌نماید.
 ۵. **بازتولیدپذیری با سخت‌افزار استاندارد:** نتایج نشان داد که با استفاده از ابزارهای بهینه‌سازی ترکیبی (آدام و L-BFGS) و دقت ممیز شناور ۶۴ بیتی، امکان بازتولید کیفی و کمی رفتارهای روش‌های فیزیک‌پایه بر روی پردازنده‌های مرکزی بدون نیاز به تجهیزات پردازش فوق‌سریع میسر است.
- به‌طور کلی، شبکه‌های آگاه از فیزیک ابزاری قدرتمند برای مسائلی هستند که داده‌های تجربی اندک بوده یا فیزیک حاکم تا حدی نامعلوم است؛ اگرچه در مسائل کلاسیک با شرایط هندسی معین، هنوز از نظر سرعت محاسباتی و دقت نهایی در جایگاهی پس از حل‌گرهای عددی پیشرفته قرار می‌گیرند.

۳-۵ پیشنهادها برای پژوهش‌های آینده

به منظور ادامه و تعمیق این مسیر پژوهشی، محورهای زیر پیشنهاد می‌گردد:

۱. **پیاده‌سازی مدل‌های زمان‌گسسته:** پیاده‌سازی و مقایسه کارایی الگوریتم‌های رانگه-کوتای ضمنی با مراحل بالا ($q > 100$) در مقایسه با مدل زمان‌پیوسته در بازه‌های زمانی طولانی.
۲. **توسعه به ابعاد بالاتر و معادلات پیچیده‌تر:** بررسی کارایی روش در حل معادلات ناویه‌استوکس دوبعدی و سه‌بعدی و ارزیابی مقیاس‌پذیری الگوریتم در هندسه‌های غیرمنظم.
۳. **روش‌های وزن‌دهی تطبیقی توابع هزینه:** به‌کارگیری راهبردهای وزن‌دهی پویا و گرادیان‌محور برای تنظیم توازن میان مولفه داده (MSE_u) و مولفه فیزیک (MSE_f) در طول آموزش.
۴. **نمونه‌برداری تطبیقی نقاط هم‌مکانی:** توسعه الگوریتم‌های نمونه‌برداری پویا بر اساس گرادیان یا باقیمانده معادله، جهت متمرکزسازی خودکار نقاط هم‌مکانی در نواحی ناپایداری و شوک.
۵. **کمی‌سازی عدم قطعیت (UQ):** ادغام چارچوب شبکه‌های آگاه از فیزیک با شبکه‌های بیزی یا رویکردهای یادگیری عمیق تلفیقی (Ensemble) جهت ارائه بازه‌های اطمینان برای پارامترهای تخمین‌زده‌شده.

فهرست منابع

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [2] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Machine learning of linear differential equations using Gaussian processes,” *Journal of Computational Physics*, vol. 348, pp. 683–693, 2017.
- [4] M. Raissi and G. E. Karniadakis, “Hidden physics models: Machine learning of nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 357, pp. 125–141, 2018.
- [5] A. Karpatne, G. Atluri, J. Faghmous, M. Steinbach, A. Banerjee, A. Ganguly, S. Shekhar, N. Samatova, and V. Kumar, “Theory-guided data science: A new paradigm for scientific discovery from data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 10, pp. 2318–2331, 2017.
- [6] M. Schmidt and H. Lipson, “Distilling free-form natural laws from experimental data,” *Science*, vol. 324, pp. 81–85, 2009.
- [7] S. L. Brunton, J. L. Proctor, and J. N. Kutz, “Discovering governing equations from data by sparse identification of nonlinear dynamical systems,” *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [8] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, “Scientific machine learning through physics-informed neural networks: Where we are and what’s next,” *Journal of Scientific Computing*, vol. 92, article 88, 2022.
- [9] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.

- [10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [11] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
- [13] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Mathematical Programming*, vol. 45, pp. 503–528, 1989.
- [14] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of AIS-TATS*, pp. 249–256, 2010.
- [15] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic differentiation in machine learning: A survey,” *Journal of Machine Learning Research*, vol. 18, pp. 1–43, 2018.
- [16] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM, 2007.
- [17] L. C. Evans, *Partial Differential Equations*, 2nd ed. American Mathematical Society, 2010.
- [18] E. Hopf, “The partial differential equation $u_t + uu_x = \mu u_{xx}$,” *Communications on Pure and Applied Mathematics*, vol. 3, pp. 201–230, 1950.
- [19] E. R. Benton and G. W. Platzman, “A table of solutions of the one-dimensional Burgers equation,” *Quarterly of Applied Mathematics*, vol. 30, pp. 195–212, 1972.
- [20] L. N. Trefethen, *Spectral Methods in MATLAB*. SIAM, 2000.
- [21] A.-K. Kassam and L. N. Trefethen, “Fourth-order time-stepping for stiff PDEs,” *SIAM Journal on Scientific Computing*, vol. 26, no. 4, pp. 1214–1233, 2005.
- [22] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Wiley, 2016.
- [23] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” in *Proceedings of OSDI*, 2016.
- [24] J. Bradbury et al., “JAX: Composable transformations of Python+NumPy programs,” 2018. [Online]. Available: github.com/google/jax

واژه‌نامه

واژه‌نامه فارسی به انگلیسی

معادل انگلیسی	واژه فارسی
Overfitting	بیش‌برازش
Backpropagation	پس‌انتشار
Activation Function	تابع فعال‌ساز
Loss Function	تابع هزینه
Initial Condition	شرط اولیه
Boundary Condition	شرط مرزی
Physics-Informed Neural Network	شبکه عصبی آگاه از فیزیک
Automatic Differentiation	مشتق‌گیری خودکار
Forward Problem	مسئله مستقیم
Inverse Problem	مسئله معکوس
Partial Differential Equation	معادله دیفرانسیل جزئی
Ordinary Differential Equation	معادله دیفرانسیل معمولی
Mean Squared Error	میانگین مربعات خطا
Learning Rate	نرخ یادگیری
Curse of Dimensionality	نفرین ابعاد
Collocation Points	نقاط هم‌مکانی

واژه‌نامه انگلیسی به فارسی

واژه فارسی	معادل انگلیسی
تابع فعال‌ساز	Activation Function
مشتق‌گیری خودکار	Automatic Differentiation
پس‌انتشار	Backpropagation
شرط مرزی	Boundary Condition
نقاط هم‌مکانی	Collocation Points
نفرین ابعاد	Curse of Dimensionality
مسئله مستقیم	Forward Problem
شرط اولیه	Initial Condition
مسئله معکوس	Inverse Problem
نرخ یادگیری	Learning Rate
تابع هزینه	Loss Function
میانگین مربعات خطا	Mean Squared Error
معادله دیفرانسیل معمولی	Ordinary Differential Equation
بیش‌برازش	Overfitting
معادله دیفرانسیل جزئی	Partial Differential Equation
شبکه عصبی آگاه از فیزیک	Physics-Informed Neural Network

پیوست الف: کد پیاده‌سازی محاسباتی

در این پیوست، متن کامل اسکریپت پیاده‌سازی پایتون بخش عملی پایان‌نامه بر بستر کتابخانه جکس (JAX) آورده شده است. این پیاده‌سازی شامل سه بخش بنیادین است:

۱. حل‌گر مرجع طیفی فوریه با گام‌زنی نمایی مرتبه چهار (ETDRK4) برای تولید داده‌های دقیق اعتبارسنجی؛
 ۲. مدل زمان‌پیوسته شبکه عصبی آگاه از فیزیک برای حل مستقیم معادله برگرز به همراه الگوریتم بهینه‌سازی ترکیبی آدام و L-BFGS؛
 ۳. فرمول‌بندی مسئله معکوس برای شناسایی ضرایب فیزیکی معادله در سناریوهای داده تمیز و نویزی.
- نسخه اجرایی این کد و کلیه داده‌ها در پوشه اختصاصی code در دسترس است.

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Physics-Informed Neural Network (PINN) for the 1D Burgers equation.
5
6 Reproduction (for B.Sc. thesis) of the continuous-time results of:
7 Raissi, Perdikaris, Karniadakis, "Physics-informed neural networks...",
8 J. Comput. Phys. 378 (2019) 686-707. (Appendix A1. and B1.
9
10 Two modes:
11 forward : data-driven solution of Burgers (learn u(t,x) from IC/BC + PDE residual)
12 inverse : data-driven discovery of Burgers (learn lambda1, lambda2 from scattered data)
13
14 Burgers equation: u_t + lambda1 * u * u_x - lambda2 * u_xx = 0
15 x in [-1, 1], t in [0, 1], lambda1 = 0.1, lambda2 = /01.0pi (forward)
16 u(0, x) = -sin(pi*x), u(t, -1) = u(t, 1) = 0
17
18 Reference solution: Fourier spectral in space (N=512, 2/3 de-aliasing) + RK4 in time.
19
20 Requirements: jax[cpu], numpy, scipy, matplotlib
21 Output: metrics JSON + figures written to ../figs/
22 """
23 import argparse
24 import json
25 import os
26 import time
27
28 import numpy as np
29
30 import jax
31 from jax import grad, jit, vmap
32 from jax.tree_util import tree_map
```

```

333 from jax.flatten_util import ravel_pytree
334 import jax.numpy as jnp
335 from jax import random
336
337 try:
338     jax.config.update("jax_enable_x64", True)
339 except Exception: # very old jax
340     from jax import config as _cfg
341     _cfg.update("jax_enable_x64", True)
342
343 import matplotlib
344 matplotlib.use("Agg")
345 import matplotlib.pyplot as plt
346
347 HERE = os.path.dirname(os.path.abspath(__file__))
348 FIGDIR = os.path.join(os.path.dirname(HERE), "figs")
349 os.makedirs(FIGDIR, exist_ok=True)
350
351 PI = float(np.pi)
352 NU = 01.0 / PI # viscosity used in the paper
353
354 # -----
355 # Reference spectral solver (independent of the neural network)
356 # -----
357
358 def burgers_reference(nx=1024, dt001.0=, t_out=None, nu=NU):
359     """Solve Burgers with Fourier spectral + ETDRK4 (Kassam & Trefethen 2005).
360
361     The diffusion term is treated exactly in Fourier space, so the scheme
362     stays stable for the stiff thin shock layer. Returns t, x, U[nt, nx].
363     """
364     L = 0.2
365     x = np.linspace(0.1-, 0.1, nx, endpoint=False)
366     k = 0.2 * PI * np.fft.fftfreq(nx, d=L / nx)
367     dealias = (np.abs(k) < (nx // 3) * 0.2) * PI / L)).astype(float)
368
369     # ETDRK4 coefficients via contour integral (M points on unit circle)
370     M = 32
371     r = np.exp(1j * PI * (np.arange(1, M + 1) - (5.0 / M)
372     Lv = -nu * k ** 2
373     LR = dt * Lv[:, None] + r[None, :]
374     E = np.exp(dt * Lv)
375     E2 = np.exp(dt * Lv / (0.2
376     Q = dt * np.real(np.mean((np.exp(LR / (0.2 - (0.1 / LR, axis=1))
377     f1 = dt * np.real(np.mean(
378         (0.4- - LR + np.exp(LR) * 0.4) - 0.3 * LR + LR ** 2)) / LR ** 3, axis=1))
379     f2 = dt * np.real(np.mean(
380         0.2 * 0.2) + LR + np.exp(LR) * (0.2- + LR)) / LR ** 3, axis=1))
381     f3 = dt * np.real(np.mean(
382         (0.4- - 0.3 * LR - LR ** 2 + np.exp(LR) * 0.4) - LR)) / LR ** 3, axis=1))
383
384     def nlin(uu):
385         uh = np.fft.fft(uu)
386         ux = np.real(np.fft.ifft(1j * k * uh))
387         return np.fft.fft(-uu * ux) * dealias
388
389     u = -np.sin(PI * x)
390     v = np.fft.fft(u)
391     if t_out is None:
392         t_out = np.linspace(0, 0.1, 101)
393     U = [u.copy()]
394     t = 0.0
395     for target in t_out[1:]:
396         nsteps = int(round((target - t) / dt))
397         for _ in range(nsteps):
398             Nv = nlin(np.real(np.fft.ifft(v)))
399             a = E2 * v + Q * Nv
400             Na = nlin(np.real(np.fft.ifft(a)))
401             b = E2 * v + Q * Na
402             Nb = nlin(np.real(np.fft.ifft(b)))
403             c = E2 * a + Q * 0.2) * Nb - Nv)
404             Nc = nlin(np.real(np.fft.ifft(c)))
405             v = E * v + f1 * Nv + f2 * (Na + Nb) + f3 * Nc

```



```

106         t += dt
107         U.append(np.real(np.fft.ifft(v)))
108     U = np.array(U)
109     assert np.all(np.isfinite(U)), "reference solver diverged!"
110     return t_out, x, U
111
112
113 # -----
114 # MLP + derivatives (automatic differentiation)
115 # -----
116 def init_mlp(key, layers):
117     keys = random.split(key, len(layers) - 1)
118     params = []
119     for kk, (din, dout) in zip(keys, zip(layers[:-1], layers[1:])):
120         W = random.normal(kk, (din, dout)) * np.sqrt(0.2) / (din + dout)
121         b = jnp.zeros((dout,))
122         params.append((W, b))
123     return params
124
125
126 def u_point(params, t, x):
127     h = jnp.stack([t, x])
128     for W, b in params[:-1]:
129         h = jnp.tanh(jnp.dot(h, W) + b)
130     W, b = params[-1]
131     return jnp.dot(h, W)[0] + b[0]
132
133
134 _u_t = grad(u_point, argnums=1)
135 _u_x = grad(u_point, argnums=2)
136 _u_xx = grad(grad(u_point, argnums=2), argnums=2)
137
138 u_pred = jit(vmap(u_point, in_axes=(None, 0, 0)))
139 _u_t_pred = jit(vmap(_u_t, in_axes=(None, 0, 0)))
140 _u_x_pred = jit(vmap(_u_x, in_axes=(None, 0, 0)))
141 _u_xx_pred = jit(vmap(_u_xx, in_axes=(None, 0, 0)))
142
143
144 # -----
145 # Manual Adam (no extra dependencies, works with any jax version)
146 # -----
147 def adam(loss_grad_fn, params, iters, lr=1e-3, log_every=500, tag="adam"):
148     m = tree_map(jnp.zeros_like, params)
149     v = tree_map(jnp.zeros_like, params)
150     b1, b2, eps = 9.0, 999.0, 1e-8
151     hist = []
152     for it in range(1, iters + 1):
153         loss, g = loss_grad_fn(params)
154         m = tree_map(lambda mm, gg: b1 * mm + (1 - b1) * gg, m, g)
155         v = tree_map(lambda vv, gg: b2 * vv + (1 - b2) * (gg ** 2), v, g)
156         mh = tree_map(lambda mm: mm / (1 - b1 ** it), m)
157         vh = tree_map(lambda vv: vv / (1 - b2 ** it), v)
158         params = tree_map(lambda p, mm, vv: p - lr * mm / (jnp.sqrt(vv) + eps),
159                           params, mh, vh)
160         hist.append(float(loss))
161         if it % log_every == 0 or it == 1:
162             print(f"[{tag}] iter {it:5d}/{iters} loss = {float(loss):6.2e}", flush=True)
163     return params, hist
164
165
166 def lbfgs(loss_fn, grad_fn, params, maxfun=2000):
167     from scipy.optimize import minimize
168     x0, unravel = ravel_pytree(params)
169     x0 = np.asarray(x0, dtype=np.float64)
170
171     def fun(x):
172         return float(loss_fn(unravel(jnp.asarray(x))))
173
174     def jac(x):
175         g = grad_fn(unravel(jnp.asarray(x)))
176         flat, _ = ravel_pytree(g)
177         return np.asarray(flat, dtype=np.float64)
178
179     res = minimize(fun, x0, jac=jac, method="LBFGS", options={"maxfun": maxfun})

```

```

189 state = {"n": 0, "hist": []}
190
191 def cb(x):
192     state["n"] += 1
193     if state["n"] % 200 == 0:
194         print(f"[lbfgs] eval {state['n']} loss = {fun(x).e}", flush=True)
195
196 t0 = time.time()
197 res = minimize(fun, x0, jac=jac, method="L-BFGS-B",
198               callback=cb,
199               options={"maxfun": maxfun, "maxiter": maxfun,
200                       "ftol": 1e-12, "gtol": 1e-10})
201 print(f"[lbfgs] done: {res.message} fun={res.fun.e} "
202       f"nit={res.nit} nfev={res.nfev} time={time.time()-t0:.f}s", flush=True)
203 return unravel(jnp.asarray(res.x, dtype=jnp.float64)), float(res.fun)
204
205 # -----
206 # Forward problem
207 # -----
208 def run_forward(adam_iters=6000, lbfgs_maxfun=6000, seed=0):
209     rng = np.random.RandomState(seed)
210     print("== forward: solving reference ==", flush=True)
211     t_g, x_g, Uref = burgers_reference()
212     print(f"reference grid: t={t_g.shape}, x={x_g.shape}, "
213           f"|u|_max at t=1: {np.abs(Uref[-1]).max().f}", flush=True)
214
215     # Training data: Nu points on initial + boundary
216     Nu0, Nub = 100, 100
217     x0 = rng.uniform(-1, 1, Nu0)
218     t0 = np.zeros_like(x0)
219     u0 = -np.sin(PI * x0)
220     tb = rng.uniform(0, 1, Nub)
221     xb = rng.choice([0.1-, [0.1, size=Nub])
222     ub = np.zeros_like(tb)
223     t_u = np.concatenate([t0, tb])
224     x_u = np.concatenate([x0, xb])
225     u_u = np.concatenate([u0, ub])
226
227     # Collocation points: half uniform + half clustered around the shock (x=0),
228     # where the solution develops a steep front for t > .3.0~
229     Nf1, Nf2 = 5000, 5000
230     t_f1 = rng.uniform(0, 1, Nf1)
231     x_f1 = rng.uniform(-1, 1, Nf1)
232     t_f2 = rng.uniform(0, 1, Nf2)
233     x_f2 = np.clip(rng.normal(0.0), 2.0, Nf2), 0.1-, (0.1
234     t_f = np.concatenate([t_f1, t_f2])
235     x_f = np.concatenate([x_f1, x_f2])
236     Nf = Nf1 + Nf2
237
238     t_u_ = jnp.asarray(t_u); x_u_ = jnp.asarray(x_u); u_u_ = jnp.asarray(u_u)
239     t_f_ = jnp.asarray(t_f); x_f_ = jnp.asarray(x_f)
240
241     layers = [2, 50, 50, 50, 50, 1]
242     params = init_mlp(random.PRNGKey(seed), layers)
243     npar = sum(w.size + b.size for w, b in params)
244     print(f"== forward: network {layers} params={npar} "
245           f"Nu={len(t_u)} Nf={Nf} ==", flush=True)
246
247     @jit
248     def loss_fn(p):
249         mse_u = jnp.mean((u_pred(p, t_u_, x_u_) - u_u_) ** 2)
250         uu = u_pred(p, t_f_, x_f_)
251         f = (u_t_pred(p, t_f_, x_f_) + uu * u_x_pred(p, t_f_, x_f_)
252              - NU * u_xx_pred(p, t_f_, x_f_))
253         mse_f = jnp.mean(f ** 2)
254         return mse_u + mse_f
255
256     @jit
257     def grad_fn(p):
258         return grad(loss_fn)(p)
259
260     def loss_grad(p):

```

```

return loss_fn(p), grad_fn(p)

print(f"[forward] initial loss = {float(loss_fn(params))6.:e}", flush=True)
t_start = time.time()
params, hist_adam = adam(loss_grad, params, adam_iters, lr=1e-3,
                          log_every=500, tag="forward/adam")
params, loss_lbfgs = lbfgs(loss_fn, grad_fn, params, maxfun=lbfgs_maxfun)
print(f"[forward] total time = {time.time()-t_start1.:f}s", flush=True)
mse_u_fin = float(jnp.mean((u_pred(params, t_u, x_u) - u_u) ** 2))
_uu = u_pred(params, t_f, x_f)
_ff = (u_t_pred(params, t_f, x_f) + _uu * u_x_pred(params, t_f, x_f)
       - NU * u_xx_pred(params, t_f, x_f))
mse_f_fin = float(jnp.mean(_ff ** 2))
print(f"[forward] final MSE_u = {mse_u_fin6.:e}, MSE_f = {mse_f_fin6.:e}",
      flush=True)

# Evaluate on the full reference grid
TT, XX = np.meshgrid(t_g, x_g, indexing="ij")
Up = np.asarray(u_pred(params, jnp.asarray(TT.ravel()),
                                         jnp.asarray(XX.ravel()))).reshape(TT.shape)
err = np.linalg.norm(Up - Uref) / np.linalg.norm(Uref)
print(f"[forward] relative L2 error = {err6.:e}", flush=True)

metrics = {
    "mode": "forward",
    "layers": layers, "n_params": int(npar),
    "Nu": int(len(t_u)), "Nf": int(Nf),
    "adam_iters": adam_iters, "lbfgs_maxfun": lbfgs_maxfun,
    "loss_adam_final": hist_adam[-1], "loss_lbfgs_final": loss_lbfgs,
    "mse_u_final": mse_u_fin, "mse_f_final": mse_f_fin,
    "rel_l2": float(err), "seed": seed,
    "nu": NU,
}

with open(os.path.join(FIGDIR, "forward_metrics.json"), "w") as f:
    json.dump(metrics, f, indent=2)
np.savez(os.path.join(FIGDIR, "forward_data.npz"),
         t=t_g, x=x_g, Uref=Uref, Upred=Up,
         hist_adam=np.array(hist_adam),
         t_u=t_u, x_u=x_u)

# --- figures ---
plt.rcParams.update({"font.size": 11})
# 1) snapshots
fig, axes = plt.subplots(1, 3, figsize=5.10, (2.3, sharey=True)
for ax, tc in zip(axes, 25.0], 5.0, [75.0:
    i = int(np.argmax(np.abs(t_g - tc)))
    ax.plot(x_g, Uref[i], "k-", lw6.1, label="Exact")
    ax.plot(x_g, Up[i], "r--", lw3.1, label="PINN")
    ax.set_title(f"t = {t_g[i2.:f]}")
    ax.set_xlabel("x")
    ax.grid(alpha(3.0=
axes[0].set_ylabel("u(t, x)")
axes[0].legend(frameon=True)
fig.suptitle("Burgers equation: exact vs PINN (forward problem)")
fig.tight_layout()
fig.savefig(os.path.join(FIGDIR, "burgers_snapshots.pdf"))
plt.close(fig)

# 2) loss history (raw curve + running minimum, as Adam is spiky)
fig, ax = plt.subplots(figsize=5.6), (8.3)
hist_adam = np.asarray(hist_adam)
runmin = np.minimum.accumulate(hist_adam)
ax.semilogy(hist_adam, color="lightsteelblue", lw8.0, label="Adam (raw)")
ax.semilogy(runmin, "b-", lw4.1, label="Adam (running min)")
ax.axhline(loss_lbfgs, color="r", ls="--", lw2.1, label="L-BFGS (final)")
ax.set_xlabel("Adam iteration")
ax.set_ylabel("Loss (MSE_u + MSE_f)")
ax.set_title("Training loss history (forward problem)")
ax.grid(alpha3.0=, which="both")
ax.legend()
fig.tight_layout()
fig.savefig(os.path.join(FIGDIR, "loss_history.pdf"))
plt.close(fig)

```

```

325 # 3) error heatmap
326 fig, ax = plt.subplots(figsize=5.6), (8.3)
327 im = ax.imshow(np.abs(Up - Uref), origin="lower", aspect="auto",
328                 extent=[x_g[0], x_g[-1], t_g[0], t_g[-1]], cmap="hot")
329 ax.set_xlabel("x")
330 ax.set_ylabel("t")
331 ax.set_title("Absolute error |u_exact - u_pinn| (forward problem)")
332 fig.colorbar(im, ax=ax, label="|error|")
333 fig.tight_layout()
334 fig.savefig(os.path.join(FIGDIR, "error_heatmap.pdf"))
335 plt.close(fig)
336
337 # 4) training points
338 fig, ax = plt.subplots(figsize=2.6), (6.3)
339 ax.scatter(t_f[:5], x_f[:5], s=3, c="lightsteelblue", label="Collocation (subset)")
340 ax.scatter(t_u, x_u, s=14, c="red", marker="x", label="IC/BC data")
341 ax.set_xlabel("t")
342 ax.set_ylabel("x")
343 ax.set_title("Training points (forward problem)")
344 ax.legend()
345 fig.tight_layout()
346 fig.savefig(os.path.join(FIGDIR, "forward_points.pdf"))
347 plt.close(fig)
348 print("[forward] figures saved.", flush=True)
349 return metrics
350
351 # -----
352 # Inverse problem (discovery of lambda1, lambda2)
353 # -----
354 def run_inverse(n_data=5000, noise0.0=, adam_iters=6000, lbfgs_maxfun=6000,
355                seed=1):
356     rng = np.random.RandomState(seed)
357     print(f"== inverse: n={n_data}, noise={noise} ==", flush=True)
358     t_g, x_g, Uref = burgers_reference()
359     # scattered observations: half uniform + half clustered around the shock,
360     # where the u_xx term (hence lambda2) actually matters.
361     n1, n2 = n_data // 2, n_data - n_data // 2
362     ti = np.concatenate([rng.uniform(0, 1, n1), rng.uniform(0, 1, n2)])
363     xi = np.concatenate([rng.uniform(-1, 1, n1),
364                          np.clip(rng.normal(0.0), 2.0, n2), 0.1-, (0.1)])
365     ii = np.clip((ti * (len(t_g) - 1)).astype(int), 0, len(t_g) - 1)
366     jj = np.clip(((xi + 1) / 2 * (len(x_g))).astype(int), 0, len(x_g) - 1)
367     ui = Uref[ii, jj].copy()
368     if noise > 0:
369         ui = ui + noise * np.std(ui) * rng.randn(n_data)
370
371     t_d = jnp.asarray(ti); x_d = jnp.asarray(xi); u_d = jnp.asarray(ui)
372
373     layers = [2, 30, 30, 30, 30, 30, 1]
374     params = init_mlp(random.PRNGKey(seed), layers)
375     lam = jnp.array(0.0), ([0.0 # lambda1, lambda2 to be identified
376     npar = sum(w.size + b.size for w, b in params) + 2
377     print(f"== inverse: network {layers} params={npar} ==", flush=True)
378
379     @jit
380     def loss_fn(pl):
381         p, l = pl
382         mse_u = jnp.mean((u_pred(p, t_d, x_d) - u_d) ** 2)
383         uu = u_pred(p, t_d, x_d)
384         f = (u_t_pred(p, t_d, x_d) + l[0] * uu * u_x_pred(p, t_d, x_d)
385              - l[1] * u_xx_pred(p, t_d, x_d))
386         return mse_u + jnp.mean(f ** 2)
387
388     @jit
389     def grad_fn(pl):
390         return grad(loss_fn)(pl)
391
392     def loss_grad(pl):
393         return loss_fn(pl), grad_fn(pl)
394
395     print(f"[inverse] initial loss = {float(loss_fn((params, lam)))6.:e}", flush=True)

```

```

t_start = time.time()
(params, lam), hist_adam = adam(loss_grad, (params, lam), adam_iters,
                                lr=1e-3, log_every=500, tag="inverse/adam")
(params, lam), loss_lbfgs = lbfgs(loss_fn, grad_fn, (params, lam),
                                maxfun=lbfgs_maxfun)
print(f"[inverse] total time = {time.time()-t_start1.:f}s", flush=True)

lam = np.asarray(lam, dtype=float)
true = np.array(0.1), NU]
pct = 0.100 * np.abs(lam - true) / np.abs(true)
print(f"[inverse] identified lambda = {lam}, true = {true}", flush=True)
print(f"[inverse] percentage errors = {pct}%", flush=True)

metrics = {
    "mode": "inverse", "layers": layers, "n_params": int(npar),
    "n_data": n_data, "noise": noise,
    "adam_iters": adam_iters, "lbfgs_maxfun": lbfgs_maxfun,
    "loss_adam_final": float(hist_adam[-1]),
    "loss_lbfgs_final": float(loss_lbfgs),
    "lambda_identified": [float(lam[0]), float(lam[1])],
    "lambda_true": 0.1, NU],
    "pct_error": [float(pct[0]), float(pct[1])],
    "seed": seed,
}

tag = f"inverse_metrics_noise{int(100*noise)}.json"
with open(os.path.join(FIGDIR, tag), "w") as f:
    json.dump(metrics, f, indent=2)

# scatter figure (only for the clean case to keep the thesis light)
if noise == :0.0
    fig, ax = plt.subplots(figsize=2.6)=, (8.3)
    sc = ax.scatter(ti, xi, s=6, c=ui, cmap="seismic", vmin=-1, vmax=1)
    ax.set_xlabel("t")
    ax.set_ylabel("x")
    ax.set_title(f"Scattered training data (inverse problem, N={n_data})")
    fig.colorbar(sc, ax=ax, label="u(t, x)")
    fig.tight_layout()
    fig.savefig(os.path.join(FIGDIR, "inverse_points.pdf"))
    plt.close(fig)
return metrics

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--mode", choices=["forward", "inverse", "all"],
                    default="all")
    ap.add_argument("--adam-iters", type=int, default=6000)
    ap.add_argument("--lbfgs-maxfun", type=int, default=6000)
    args = ap.parse_args()

    all_metrics = {}
    if args.mode in ("forward", "all"):
        all_metrics["forward"] = run_forward(
            adam_iters=args.adam_iters, lbfgs_maxfun=args.lbfgs_maxfun)
    if args.mode in ("inverse", "all"):
        all_metrics["inverse_clean"] = run_inverse(
            noise0.0=, adam_iters=args.adam_iters,
            lbfgs_maxfun=args.lbfgs_maxfun)
        all_metrics["inverse_noisy"] = run_inverse(
            noise01.0=, adam_iters=args.adam_iters,
            lbfgs_maxfun=args.lbfgs_maxfun, seed=2)
    with open(os.path.join(FIGDIR, "metrics_all.json"), "w") as f:
        json.dump(all_metrics, f, indent=2, default=str)
    print("ALL DONE. metrics:", json.dumps(all_metrics, indent=2, default=str),
          flush=True)

if __name__ == "__main__":
    main()

```

فرم ارزیابی پایان نامه کارشناسی

نام و نام خانوادگی دانشجو	محمد امیر بابازاد
شماره دانشجویی	۱۴۰۲۱۳۵۱۰۴
رشته تحصیلی	علوم کامپیوتر
عنوان پایان نامه	شبکه‌های عصبی آگاه از فیزیک برای حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی
استاد راهنما	دکتر بابک آذرنوید
تاریخ دفاع	
نمره به عدد	
نمره به حروف	
نام و امضای استاد راهنما	
نام و امضای داور	
نام و امضای مدیر گروه	

Abstract

The numerical solution of nonlinear partial differential equations (PDEs) is a fundamental challenge in scientific computing and engineering. While classical numerical methods are mathematically rigorous, they inherently rely on computational mesh generation and suffer from the curse of dimensionality. On the other hand, purely data-driven deep learning models require vast training datasets and lack guarantees regarding physical consistency. In recent years, the physics-informed neural network (PINN) framework has emerged as a promising paradigm that seamlessly integrates governing physical laws into deep learning architectures. This thesis presents a comprehensive study, implementation, and empirical evaluation of PINNs for solving both forward and inverse problems in nonlinear PDEs.

First, we establish the theoretical foundations, including partial differential equations, classical numerical schemes, deep feedforward networks, optimization algorithms, and automatic differentiation. We then formulate the continuous- and discrete-time PINN frameworks, detailing the construction of residual networks and composite loss functions. In the computational section, the continuous-time model is implemented for the one-dimensional viscous Burgers equation in Python using the JAX library. An independent high-precision reference solver based on the Fourier spectral method with fourth-order exponential time-differencing (ETDRK4) is developed and verified against the semi-analytical Cole-Hopf solution.

In the forward problem, the continuous solution across the spatio-temporal domain is reconstructed using only 200 initial and boundary data points, achieving a relative L_2 error of 2.15×10^{-2} compared to the spectral reference; spatial error analysis demonstrates that the error is confined to the sharp shock layer. In the inverse problem, unknown advection and diffusion coefficients are identified from 5000 scattered observations: the advection coefficient is discovered with an error of 8.8% and diffusion with 32.6% in the noise-free case, and with errors of 7.7% and 24.9% respectively under one percent Gaussian noise. The results confirm the exceptional data efficiency and robustness of physics-informed neural networks.

Keywords: Physics-Informed Neural Networks, Nonlinear Partial Differential Equations, Forward Problem, Inverse Problem, Burgers Equation, Automatic Differentiation, Fourier Spectral Method.



University of Bonab

Department of Computer Science

B.Sc. Thesis in Computer Science

**Physics-Informed Neural Networks
for Solving Forward and Inverse
Problems
Involving Nonlinear Partial
Differential Equations**

Supervisor: Dr. Babak Azarnavid

By: Mohammad Amir Babazad

Student ID: 1402135104

September 2026